

Retrieval Augmented Generation (RAG) for Everyone!

What is RAG	2
RAG Components	3
Advantages of Retrieval Augmented Generation	3
Systematic RAG Workflow	4
RAG Retrieval Sources	6
RAG Tutorial	7
Evolution of RAG Over Time	8
RAG Design Choices	9
Chunking Strategies in RAG	10
RAG Using LangChain	12
Advanced RAG	13
Reranking in RAG	14
Types of Embedding Models for RAG	16
Semantic Chunking in RAG Applications	18
Retrieval Pain Points in RAG	19
RAG Enhancement Techniques	21
RAG Best Practices	23
Semantic Cache to Improve RAG	24
Improving RAG Pipeline	26
Metrics for RAG Performance	27
RAG Approaches	29
Advanced RAG Techniques	30
Advanced RAG with MultiQuery Retriever	31
Custom RAG Chatbot	33
Robust and Safe RAG Overview	34
Implementing RAG Using LangChain and SingleStore	36
Modular RAG Framework	37
Adaptive RAG	38
Advanced RAG Using LlamaIndex and Claude 3	40
Advanced RAG Using RAPTOR	41
Agentic RAG Using LlamaIndex	43
Building a Multimodal RAG Workflow	44
Agentic RAG Using CrewAI & LangChain	45
Live RAG Comparison with Different Vector Databases	46
RAG Setup Evaluation Using LlamaIndex	48

Production Ready RAG Pipelines	49
RAG Using Llama 3.1 Model	49
Verifying the Correctness of RAG Responses	50
RAG with Knowledge Graphs	51
Vector RAG vs. Graph RAG	52
RAG Evaluation Strategies	53
BUT....RAG Isn't a Silver Bullet :(	55

New to the world of Retrieval Augmented Generation (RAG)? We've got you covered with this in-depth guide.

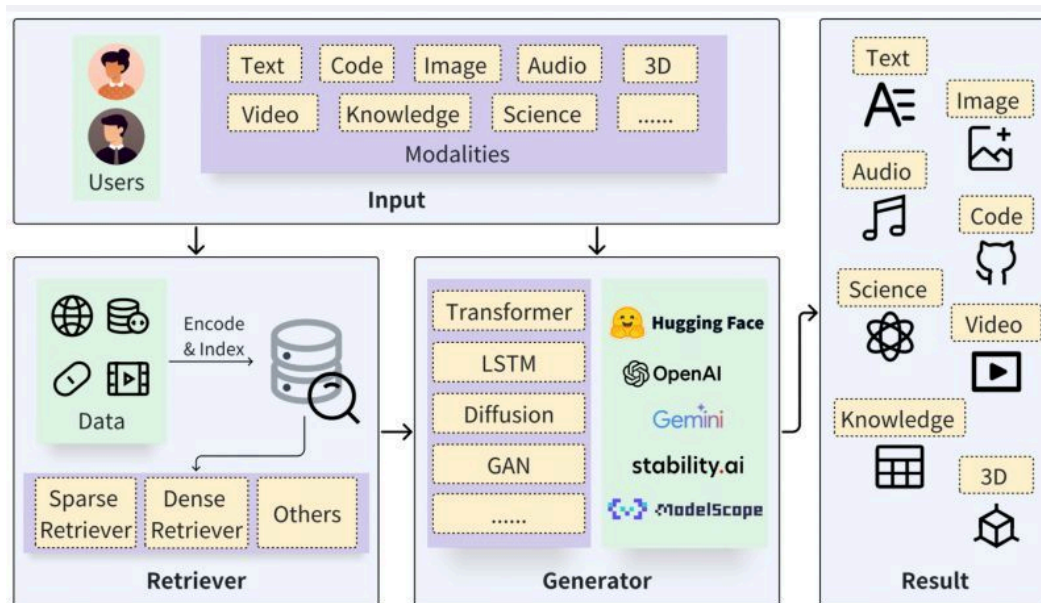
Large language models (LLMs) are becoming the backbone of most of the organizations these days as the whole world is making the transition towards AI. While LLMs are all good and trending for all the positive reasons, they also pose some disadvantages if not used properly. Yes, LLMs can sometimes produce the responses that aren't expected, they can be fake, made up information or even biased. Now, this can happen for various reasons. We call this process of generating misinformation by LLMs as hallucination.

There are some notable approaches to mitigate the LLM hallucinations such as fine-tuning, prompt engineering, retrieval augmented generation (RAG) etc. Retrieval augmented generation (RAG) has been the most talked about approach in mitigating the hallucinations faced by large language models. Today we will see everything about the RAG approach, what it is, how it works, its components, workflow from basic to advanced.

What is RAG

Retrieval-Augmented Generation (RAG) is a natural language processing framework that enhances large language models (LLMs) by integrating external data retrieval with text generation. It retrieves relevant information from external sources/databases/custom source to improve response accuracy and relevance, mitigating issues like misinformation and outdated knowledge in generated content. So, RAG basically reduces the LLM hallucinations by providing contextually relevant responses through the data sources provided/attached.

RAG Components



The RAG pipeline basically involves three critical components: Retrieval component, Augmentation component, Generation component.

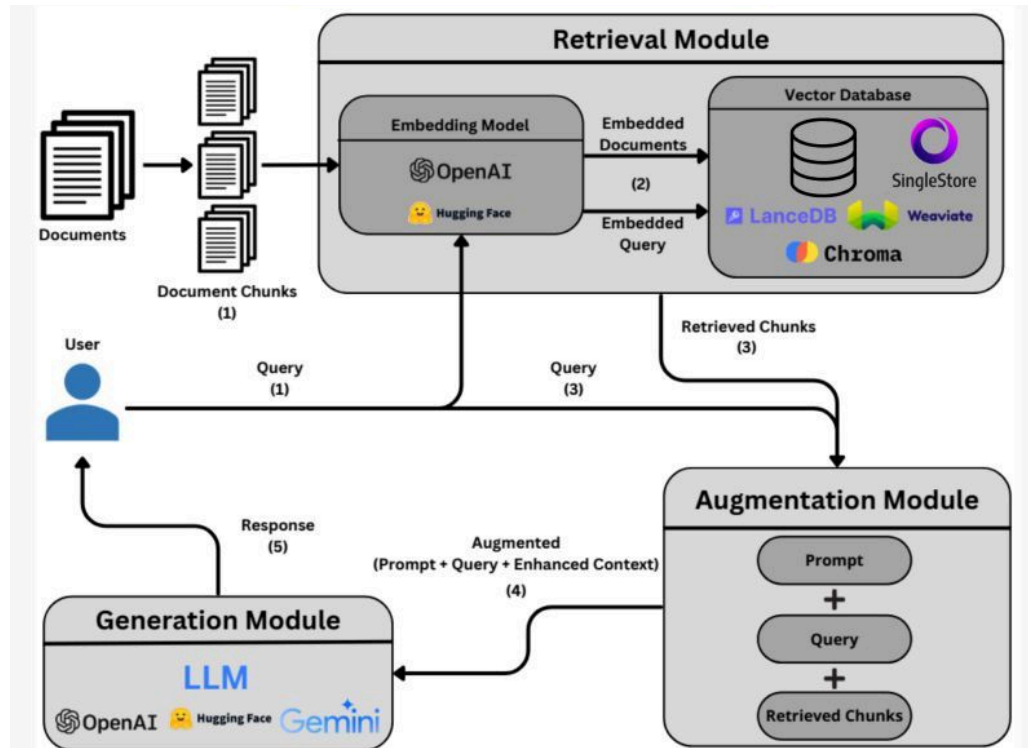
- **Retrieval:** This component helps you fetch the relevant information from the external knowledge base like a vector database for any given user query. This component is very crucial as this is the first step in curating the meaningful and contextually correct responses.
- **Augmentation:** This part involves enhancing and adding more relevant context to the retrieved response for the user query.
- **Generation:** Finally, a final output is presented to the user with the help of a large language model (LLM). The LLM uses its own knowledge and the provided context and comes up with an apt response to the user's query.

Advantages of Retrieval Augmented Generation

There are some incredible advantages of RAG. Let me share some notable ones:

- **Scalability.** RAG approach helps you with scale models by simply updating or adding external/custom data to your external database (vector database).
- **Memory efficiency.** Traditional models like GPT have limits when it comes to pulling fresh and updated information and fails to be memory efficient. RAG leverages external databases like a vector database — allowing it to pull in fresh, updated or detailed information when needed with speed.
- **Flexibility.** By updating or expanding the external knowledge source, you can adapt RAG to build any AI applications with flexibility.

Systematic RAG Workflow



RAG consists of three modules that you need to understand!

Retrieval module, Augmentation module, and Generation module (as discussed above).

First, the document which forms the source database is divided into chunks. These chunks, transformed into vectors using an embedding model like OpenAI or open source models available from Hugging Face community, are then embedded into a high-dimensional vector database (e.g., SingleStore Database, Chroma and LlamaIndex).

When the user inputs a query, the query is embedded into a vector using the same embedding model. Then, chunks whose vectors are closest to the query vector, based on some similarity metrics (e.g., cosine similarity) are retrieved. This process is contained in the retrieval module shown in the figure. After that, the retrieved chunks are augmented to the user's query and the system prompt in the augmentation module.

This step is critical for making sure that the records from the retrieved documents are effectively incorporated with the query. Then, the output from the augmentation module is fed to the generation module which is responsible for generating an accurate answer to the query by utilizing the retrieved chunks and the prompt through an LLM (like chatGPT by OpenAI, hugging face, and Gemini by Google).

But to make RAG work perfectly, here are some key points to consider:

1. **Quality of External Knowledge Source:** The quality and relevance of the external knowledge source used for retrieval are crucial.

2. **Embedding Model:** The choice of the embedding model used for retrieving relevant documents or passages from the knowledge source is important.

3. **Chunk Size and Retrieval Strategy:** Experiment with different chunk sizes to find the optimal length for context retrieval. Larger chunks may provide more context but could also introduce irrelevant information. Smaller chunks may focus on specific details but might lack broader context.

4. **Integration with Language Model:** The way the retrieved information is integrated with the language model's generation process is crucial. Techniques like cross-attention or memory-augmented architectures can be used to effectively incorporate the retrieved information into the model's output.

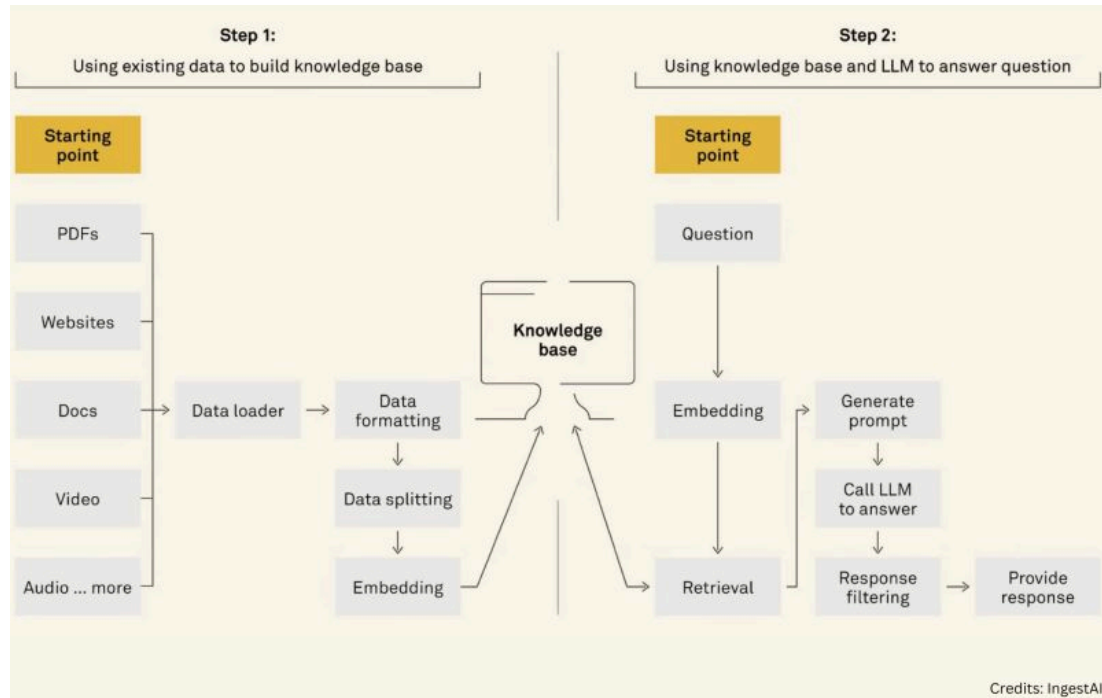
5. **Evaluation and Fine-tuning:** Evaluating the performance of the RAG model on relevant datasets and tasks is important to identify areas for improvement. Fine-tuning the RAG model on domain-specific or task-specific data can further enhance its performance.

6. **Ethical Considerations:** Ensure that the external knowledge source is unbiased and does not contain offensive or misleading information.

7. **Handling Out-of-Date or Incorrect Information:** It's important to have strategies in place for handling situations where the retrieved information is out-of-date or incorrect.

Use SingleStore Database as your vector store, try for free: <https://bit.ly/SingleStoreDB>

RAG Retrieval Sources



Do you know how RAG applications acquire external knowledge?

RAG systems can leverage various types of retrieval sources to acquire external knowledge.

The most common data types include:

- ☐ **Unstructured Data (Text):** This includes plain text documents, web pages, and other free-form textual sources.
- ☐ **Semi-Structured Data (PDF):** PDF documents, such as research papers, reports, and manuals, contain a mix of textual and structural information.
- ☐ **Structured Data (Knowledge Graphs):** Knowledge graphs, such as Wikipedia and Freebase, represent information in a structured and interconnected format.
- ☐ **LLM-Generated Content:** Recent advancements have shown that LLMs themselves can generate high-quality content that can be used as a retrieval source. This approach leverages the knowledge captured within the LLM's parameters to generate relevant information.

All this data gets converted into embeddings and gets stored in a vector database. When a user query comes in, it also gets converted into an embedding (query embedding) and the most relevant answer will be retrieved using semantic search. The vector database becomes knowledge base to search for the contextually relevant answer.

Additionally, one more aspect to consider is retrieval granularity. It refers to the level at which knowledge is retrieved from the sources.

Common levels of retrieval granularity include:

- ❑ **Phrase-Level Retrieval:** This involves retrieving short phrases or snippets of text that are highly relevant to the query. Phrase-level retrieval can provide precise and targeted information but may lack broader context.
- ❑ **Sentence-Level Retrieval:** Sentence-level retrieval focuses on retrieving complete sentences that contain relevant information. It strikes a balance between specificity and context, making it suitable for a wide range of tasks.
- ❑ **Chunk-Level Retrieval:** Chunk-level retrieval involves retrieving larger chunks of text, such as paragraphs or sections. It provides more comprehensive information and context but may introduce noise and irrelevant details.
- ❑ **Document-Level Retrieval:** Document-level retrieval retrieves entire documents that are relevant to the query. While it offers the most extensive context, it may require additional processing to extract the most pertinent information.

Know more about knowledge retrieval in RAG: <https://ingestai.io/blog/knowledge-retrieval-in-rag>

RAG Tutorial

Let's build a simple AI application that can fetch the contextually relevant information from our own data for any given user query.

[Follow the complete hands-on tutorial from my Medium article.](#)

Implementing Retrieval Augmented Generation (RAG): A Hands-On Guide!

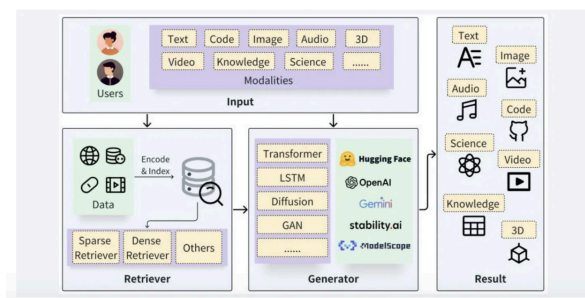


Pavan Belagatti

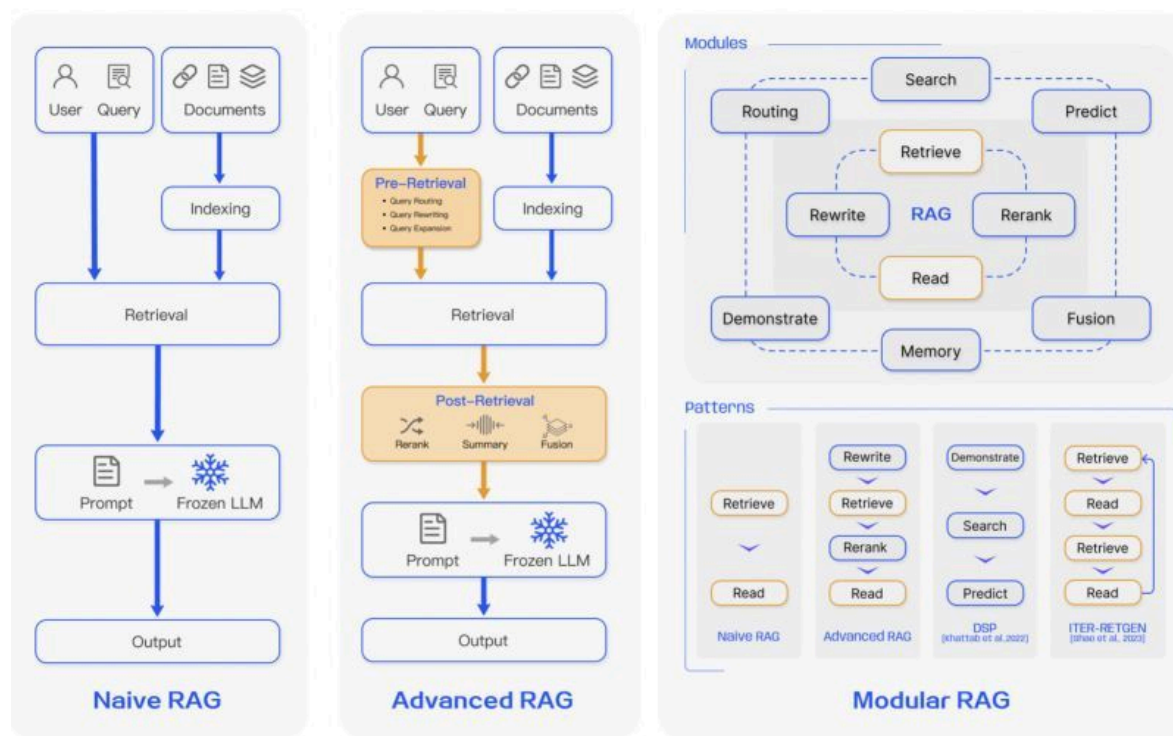
Published in Generative AI · 7 min read · Apr 18, 2024

369

4



Evolution of RAG Over Time



Let's talk about the RAG evolution over time.

1. Naive RAG:

The Naive RAG research paradigm represents the earliest methodology, which gained prominence shortly after the widespread adoption of ChatGPT. The Naive RAG follows a traditional process that includes indexing, retrieval, and generation. It is also characterized as a “Retrieve-Read” framework [Ma et al., 2023a].

2. Advanced RAG:

Advanced RAG has been developed with targeted enhancements to address the shortcomings of Naive RAG. In terms of retrieval quality, Advanced RAG implements pre-retrieval and post-retrieval strategies. To address the indexing challenges experienced by Naive RAG, Advanced RAG has refined its indexing approach using techniques such as sliding window, fine-grained segmentation, and metadata. It has also introduced various methods to optimize the retrieval process [ILIN, 2023].

3. Modular RAG:

The modular RAG structure diverges from the traditional Naive RAG framework, providing greater versatility and flexibility. It integrates various methods to enhance functional modules, such as incorporating a search module for similarity retrieval and applying a fine-tuning approach in the retriever [Lin et al., 2023].

Restructured RAG modules [Yu et al., 2022] and iterative methodologies like [Shao et al., 2023] have been developed to address specific issues. The modular RAG paradigm is increasingly becoming the norm in the RAG domain, allowing for either a serialized pipeline or an end-to-end training approach across multiple modules.

This comprehensive review paper offers a detailed examination of the progression of RAG paradigms, encompassing the Naive RAG, the Advanced RAG, and the Modular RAG.

Access the paper here: <https://arxiv.org/abs/2312.10997>

RAG Design Choices

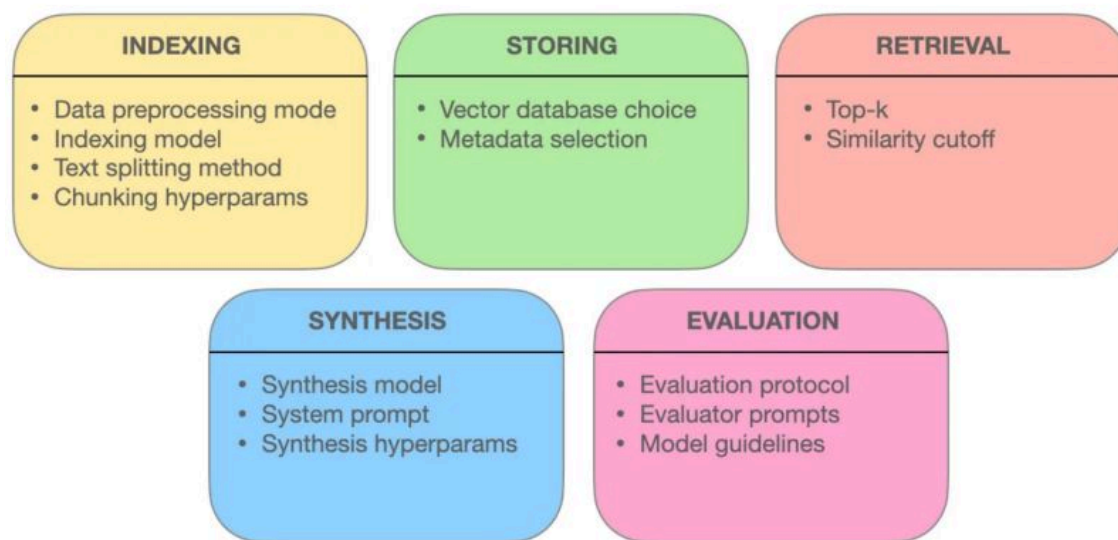


Image credits: Michał Oleszak

Let's discuss some useful list of RAG design choices.

RAG basically consists of five main pieces/components:

- ➡ Indexing: Embedding external data into a vector representation.
- ➡ Storing: Persisting the indexed embeddings in a database.
- ➡ Retrieval: Finding relevant pieces in the stored data.
- ➡ Synthesis: Generating answers to user's queries.
- ➡ Evaluation: Quantifying how good the RAG system is.

When designing the indexing step, there are a few design choices to make:

- Data processing mode
- Indexing model
- Text splitting method
- Chunking hyperparameters

The best embedding models might be different than the best LLMs in general.

When designing the storing step of a RAG pipeline, the two most important decisions are:

- Database choice
- Metadata selection

Sometimes finding a vector database might be very confusing due to so many databases available today. SingleStore database started supporting vector storage long back in 2017 itself. I would highly recommend choosing SingleStore as your vector database for all your AI/ML applications.

[Try SingleStore for free: <https://bit.ly/SingleStoreDB>]

There are a few things you would need to think about when designing the retrieval step:

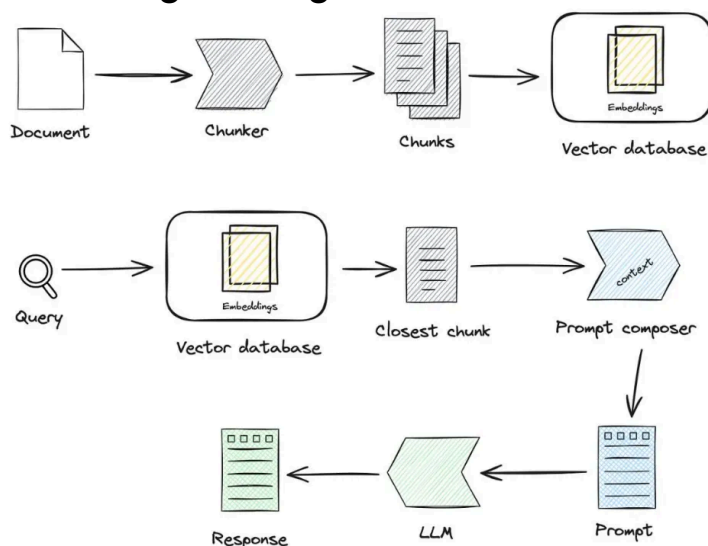
- Retrieval strategy
- Retrieval hyperparameters
- Query transformations

The most important aspects of the evaluation step are:

- Evaluation protocol
- Evaluator prompts
- Model guidelines

Know in detail about each step and useful considerations in this original guide: <https://towardsdatascience.com/designing-rags-dbb9a7c1d729>

Chunking Strategies in RAG



Improving the efficiency of LLM applications via RAG is all great.

BUT the question is, what should be the right chunking strategy?

Chunking is the method of breaking down the large files into more manageable segments/chunks so the LLM applications can get proper context and the retrieval can be easy. In a video on YouTube, Greg Kamradt provides overview of different chunking strategies. Let's understand them one by one.

They have been classified into five levels based on the complexity and effectiveness.

❑ **Level 1 : Fixed Size Chunking**

This is the most crude and simplest method of segmenting the text. It breaks down the text into chunks of a specified number of characters, regardless of their content or structure. Langchain and llamaindex framework offer CharacterTextSplitter and SentenceSplitter (default to splitting on sentences) classes for this chunking technique.

❑ **Level 2: Recursive Chunking**

While Fixed size chunking is easier to implement, it doesn't consider the structure of text. Recursive chunking offers an alternative. In this method, we divide the text into smaller chunk in a hierarchical and iterative manner using a set of separators. Langchain framework offers RecursiveCharacterTextSplitter class, which splits text using default separators (“\n\n”, “\n”, “, ”)

❑ **Level 3 : Document Based Chunking**

In this chunking method, we split a document based on its inherent structure. This approach considers the flow and structure of content but may not be as effective documents lacking clear structure.

❑ **Level 4: Semantic Chunking**

All above three levels deals with content and structure of documents and necessitate maintaining constant value of chunk size. This chunking method aims to extract semantic meaning from embeddings and then assess the semantic relationship between these chunks. The core idea is to keep together chunks that are semantic similar. Llamaindex has SemanticSplitterNodeParser class that allows to split the document into chunks using contextual relationship between chunks.

❑ **Level 5: Agentic Chunking**

This chunking strategy explore the possibility to use LLM to determine how much and what text should be included in a chunk based on the context.

Know more about these chunking strategies in [this article](#).

RAG Using LangChain

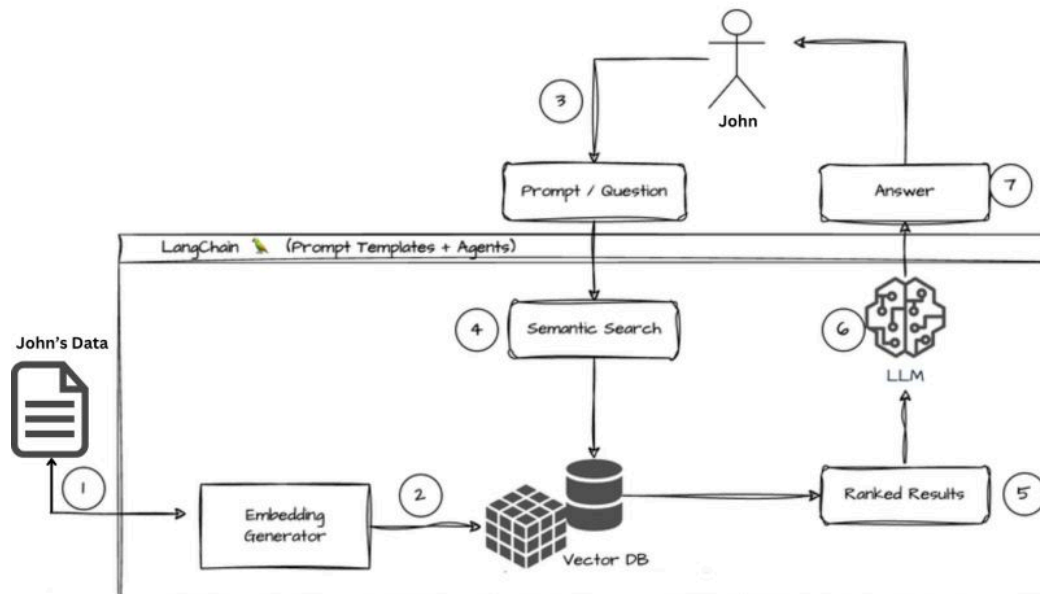


Image Credits: Cisco Community

LangChain is a powerful framework for LLM-powered applications.

1. It provides a standard interface for chains, enabling developers to create sequences of calls that go beyond a single LLM call.
2. Langchain allows developers to build chatbots, generative question-answering systems, summarization tools, and more.
3. It simplifies the process of working with LLMs and provides tools for prompt management, memory, indexing, and agent-based decision-making.
4. Langchain is designed to be data-aware and agentic, connecting language models to other data sources and allowing them to interact with their environment.

Let's see the how RAG works using LangChain.

1. Documents are converted into a vector representation, often referred to as an embedding.
2. Embeddings (vectorized documents) are stored in a vector database
3. The user asks a question.
4. Once the data is stored in the database, Langchain supports various retrieval algorithms. These include basic semantic search, parent document retriever, self-query retriever, ensemble retriever, and more.

5. When conducting a search, the retrieval system assigns a score or ranking to each document based on its relevance to the query.

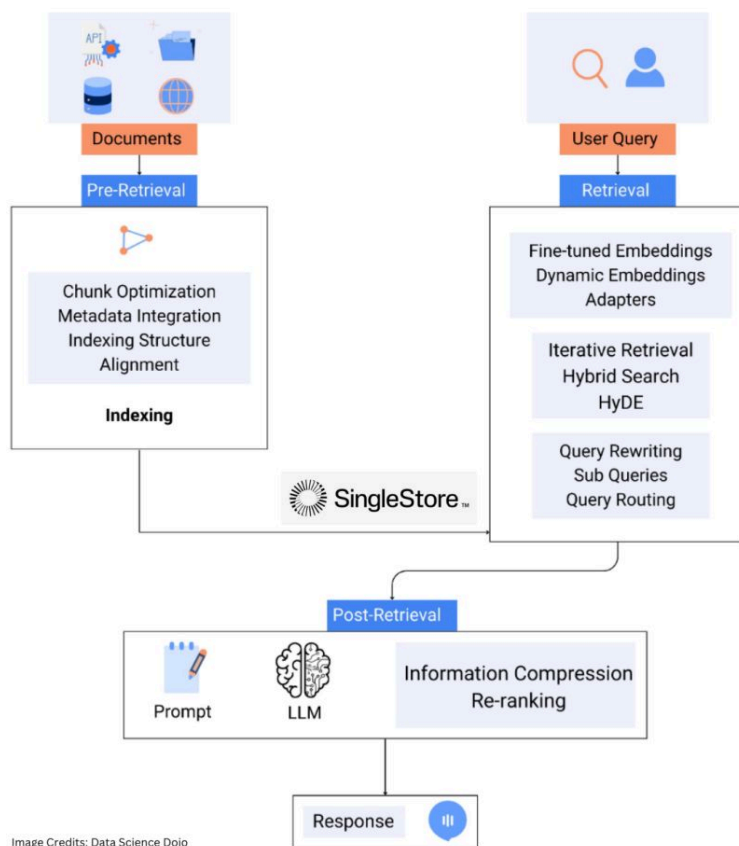
6. Results are sent to the LLM

7. Leveraging the contextual representation, the model then generates a response.

Wanna do a hands-on tutorial?

Here is my guide on implementing RAG using LangChain: A Step-by-Step Guide - <https://levelup.gitconnected.com/implementing-rag-using-langchain-and-singlestore-a-step-by-step-guide-2a579da1de0c>

Advanced RAG



Let's use some simple query examples from the basic RAG explanation: "What's the latest breakthrough in renewable energy?", to better understand these advanced techniques.

□ **Pre-retrieval optimizations:** Before the system begins to search, it optimizes the query for better outcomes. For our example, Query Transformations and Routing might break down the query into sub-queries like "latest renewable energy breakthroughs" and "new technology in renewable energy."

This ensures the search mechanism is fine-tuned to retrieve the most accurate and relevant information.

□ **Enhanced retrieval techniques:** During the retrieval phase, Hybrid Search combines keyword and semantic searches, ensuring a comprehensive scan for information related to our query. Moreover, by Chunking and Vectorization, the system breaks down extensive documents into digestible pieces, which are then vectorized.

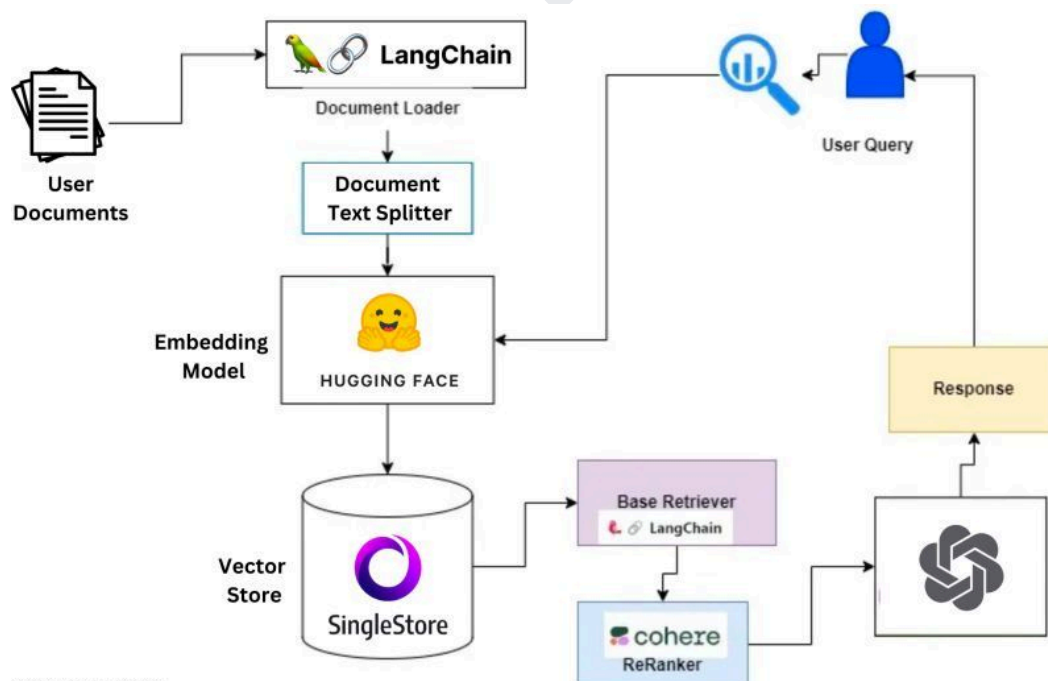
This means our query doesn't just pull up general information but seeks out the precise segments of texts discussing recent innovations in renewable energy.

□ **Post-retrieval refinements:** After retrieval, Reranking and Filtering processes evaluate the gathered information chunks. Instead of simply using the top 'k' matches, these techniques rigorously assess the relevance of each piece of retrieved data. For our query, this could mean prioritizing a segment discussing a groundbreaking solar panel efficiency breakthrough over a more generic update on solar energy.

This step ensures that the information used in generating the response directly answers the query with the most relevant and recent breakthroughs in renewable energy.

Know more in the original article: <https://datasciencedojo.com/blog/rag-vs-finetuning-llm-debate/>

Reranking in RAG



Credits: Plaban Nayak

Traditional semantic search consists of a two-part process.

First, an initial retrieval mechanism does an approximate sweep over a collection of documents and creates a document list.

Then, a re-ranker mechanism will take this candidate document list and re-rank the elements. With Rerank, we can improve your models by re-organizing your results based on certain parameters.

Why is Re-Ranking Required ?

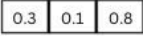
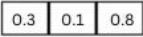
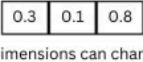
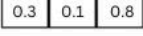
- ❑ The recall performance for LLMs decreases as we add more context resulting in increased context window(context stuffing)
- ❑ Basic Idea behind reranking is to filter down the total number of documents into a fixed number .
- ❑ The re-ranker will re-rank the records and get the most relevant items at the top and they can be sent to the LLM
- ❑ The Reranking offers a solution by finding those records that may not be within the top 3 results and put them into a smaller set of results that can be further fed into the LLM

Reranking basically enhance the relevance and precision of retrieved results.

Know more in this article:

<https://medium.aiplanet.com/advanced-rag-cohere-re-ranker-99acc941601c>

Types of Embedding Models for RAG

Model type	Embeddings	Example model	Compute	Granularity/input
Sparse		SPLADE	Low	Sentence, paragraph
Dense		Sentence transformers	Medium	Sentence, paragraph
Multivector		COLBERT	High	Sentence, paragraph
Long context dense		text-embedding-3-small	Medium	Paragraphs, chapters
Variable dimension		text-embedding-3-small	Medium	Sentence, paragraph
Code (dense)		text-embedding-3-small	Medium	Functions, classes

Credits: Pratik Bhavsar, Galileo Labs

How to Select an Embedding Model for Your RAG Application?

Embeddings form the foundation for achieving precise and contextually relevant LLM outputs across different tasks.

Which encoder you select to generate embeddings is a critical decision, hugely impacting the overall success of the RAG system. Low quality embeddings lead to poor retrieval.

When selecting an embedding model, consider the vector dimension, average retrieval performance, and model size.

Companies such as OpenAI, Cohere, and Voyage consistently release enhanced embedding models.

Different types of embeddings are designed to address unique challenges and requirements in different domains.

□ Dense embeddings are continuous, real-valued vectors that represent information in a high-dimensional space. In the context of RAG applications, dense embeddings, such as those generated by models like OpenAI's Ada or sentence transformers, contain non-zero values for every element.

- ❑ Sparse embeddings, on the other hand, are representations where most values are zero, emphasizing only relevant information. In RAG applications, sparse vectors are essential for scenarios with many rare keywords or specialized terms.
- ❑ Multi-vector embedding models like ColBERT feature late interaction, where the interaction between query and document representations occurs late in the process, after both have been independently encoded.
- ❑ Long documents have always posed a particular challenge for embedding models. The limitation on maximum sequence lengths, often rooted in architectures like BERT, leads to practitioners segmenting documents into smaller chunks. Unfortunately, this segmentation can result in fragmented semantic meanings and misrepresentation of entire paragraphs.
- ❑ Variable dimension embeddings are a unique concept built on Matryoshka Representation Learning (MRL). MRL learns lower-dimensional embeddings that are nested into the original embedding, akin to a series of Matryoshka Dolls.
- ❑ Code embeddings are a recent development used to integrate AI-powered capabilities into Integrated Development Environments (IDEs), fundamentally transforming how developers interact with codebases.

There are several factors that need to be considered while selecting an embedding model.

Know more about embeddings and models in this article:

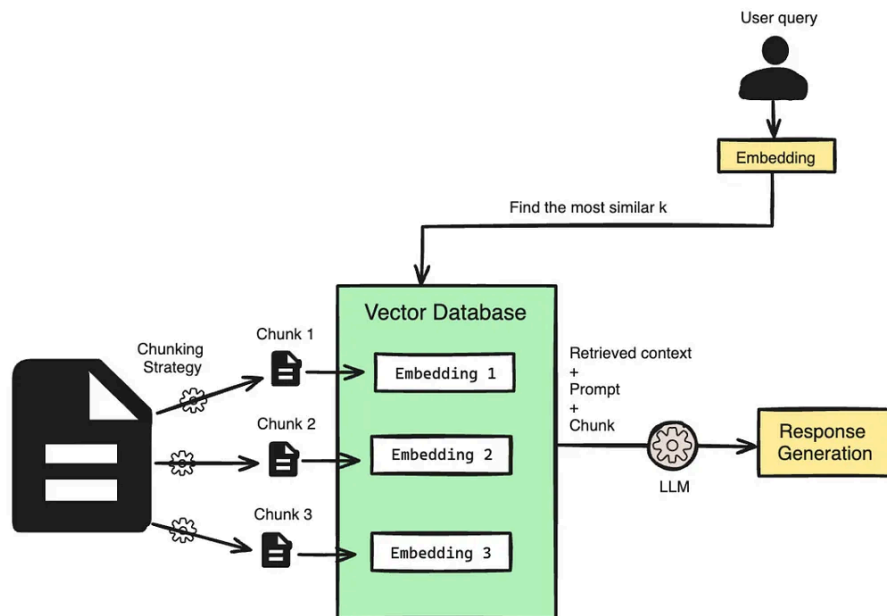
<https://www.rungalileo.io/blog/mastering-rag-how-to-select-an-embedding-model>

No matter which embedding model you use, having a robust database is a must for your RAG application.

Use SingleStore as your vector database to build your AI/ML apps. Sign up & use for free:

<https://bit.ly/SingleStoreDB>

Semantic Chunking in RAG Applications



Chunking in RAG applications involves breaking down large pieces of data into smaller, manageable segments or “chunks.” This process enhances the efficiency and accuracy of information retrieval by enabling the model to handle more precise and relevant portions of data.

In RAG systems, when a query is made, the model searches through these chunks to find the most relevant information, rather than going through an entire document.

This not only speeds up the retrieval process but also improves the quality of the generated responses by focusing on the most pertinent information.

Chunking is especially useful in scenarios where documents are lengthy or contain diverse topics, as it ensures that the retrieved data is contextually appropriate and precise.

Naive chunking strategies limit themselves with dividing the text into chunks of a fixed number of words or characters, and not always effective.

Semantic Chunking is a method that focuses on extracting and preserving the semantic meaning within text segments. By utilizing embeddings to capture the underlying semantics, this approach assesses the relationships between different chunks to ensure that similar content is kept together.

By focusing on the text’s meaning and context, Semantic Chunking significantly enhances retrieval quality. It’s ideal for maintaining semantic integrity, ensuring coherent and relevant information retrieval.

Let's see how semantic chunking is better than your naive chunking strategies in my tutorial: <https://levelup.gitconnected.com/semantic-chunking-for-enhanced-rag-applications-b6bc92942af0>

Retrieval Pain Points in RAG

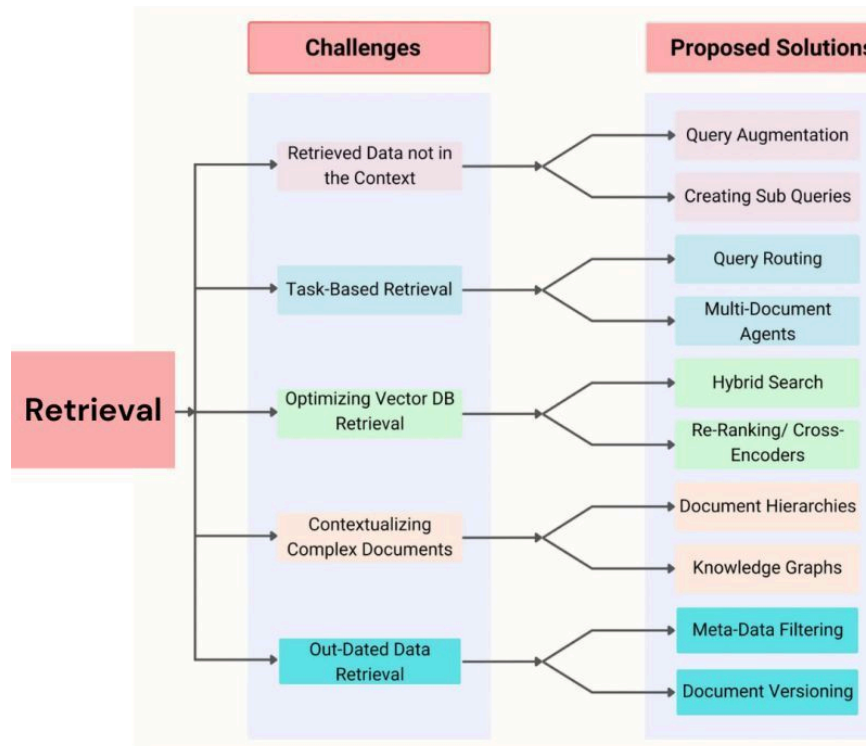


Image Credits: Data Science Dojo

The approach of RAG might not be as easy as you think.

Effective retrieval is a pain, and you can encounter several issues during this important stage.

Here are some common pain points and possible solutions in the retrieval stage.

□ Challenge: Retrieved data not in context & there can be several reasons for this.

- Missed Top Rank Documents: The system sometimes doesn't include essential documents that contain the answer in the top results returned by the system's retrieval component.
- Incorrect Specificity: Responses may not provide precise information or adequately address the specific context of the user's query
- Losing Relevant Context During Reranking: This occurs when documents containing the answer are retrieved from the database but fail to make it into the context for generating an answer.

□ Proposed Solutions:

- Query Augmentation: Query augmentation enables RAG to retrieve information that is in context by enhancing the user queries with additional contextual details or modifying them to maximize relevancy. This involves improving the phrasing, adding company-specific context, and generating sub-questions that help contextualize and generate accurate responses
 - Rephrasing
 - Hypothetical document embeddings
 - Sub-queries
- Tweak retrieval strategies: Llama Index offers a range of retrieval strategies, from basic to advanced, to ensure accurate retrieval in RAG pipelines. By exploring these strategies, developers can improve the system's ability to incorporate relevant information into the context for generating accurate responses.
 - Small-to-big sentence window retrieval,
 - recursive retrieval
 - semantic similarity scoring.
- Hyperparameter tuning for chunk size and similarity_top_k: This solution involves adjusting the parameters of the retrieval process in RAG models. More specifically, we can tune the parameters related to chunk size and similarity_top_k.

The chunk_size parameter determines the size of the text chunks used for retrieval, while similarity_top_k controls the number of similar chunks retrieved. By experimenting with different values for these parameters, developers can find the optimal balance between computational efficiency and the quality of retrieved information.

- Reranking: Reranking retrieval results before they are sent to the language model has proven to improve RAG systems' performance significantly.

This reranking process can be implemented by incorporating the reranker as a postprocessor in the RAG pipeline.

Know more about the other pain points & possible solutions explained in detail:

<https://datasciencedojo.com/blog/rag-challenges-in-llm-applications/>

RAG Enhancement Techniques

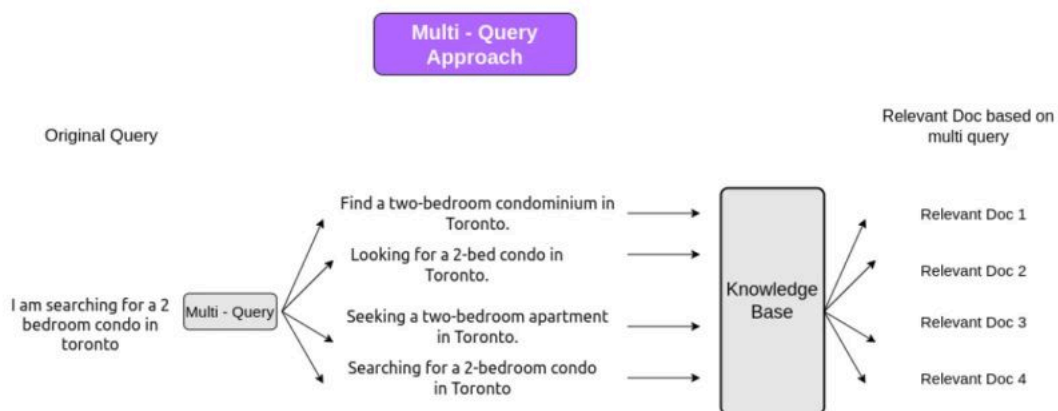
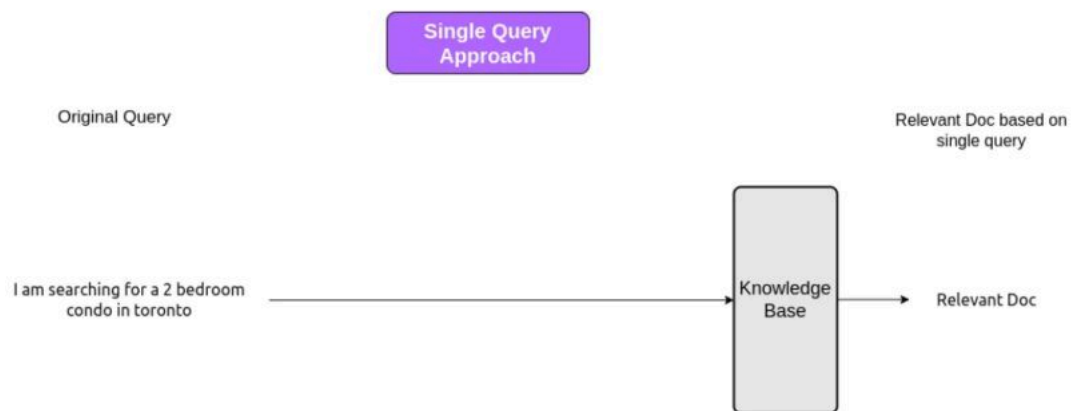


Image Credits: BavalpreetSinghh

The road to building RAG applications is not a smooth one.

You need to know some techniques to overcome different challenges that RAG throws at you while building LLM powered applications.

1. Transformation from Single Query to Multi Query:

Multi-Query is an advanced approach in the Query Transformation stage of retrieval. Unlike traditional methods where only one query is used, Multi-Query generates multiple queries and retrieves similar documents for each one. Builders utilize Multi-Query primarily for two reasons: enhancing suboptimal queries and expanding result sets. It addresses users' imperfect queries by filling in gaps and retrieves more diverse results, leading to an expanded results set that can provide better answers than single-query documents.

2. Improving Indexed Data Quality:

Unfortunately, data cleaning is often overlooked during the development of RAGs, with a tendency to ingest all available documents without verifying their quality. We need to ensure that the data fed into the RAG system is of high quality for obtaining accurate answers. The principle of “garbage in, garbage out” is especially relevant here.

3. Chunking strategy and size matters to optimize index structure:

When setting up your Retrieval Augmented Generation (RAG) system, the size of the chunks and chunking technique plays a crucial role. It determines how much information is retrieved from the document store for processing. Choosing a small chunk size may lead to missing important details, while opting for a larger size could introduce irrelevant information.

4. Incorporation of metadata with indexed vectors:

Adding metadata alongside indexed vectors in the vector database offers significant benefits in organizing and enhancing search relevance.

5. Improving search relevance with question-based indexing:

LLMs and RAGs offer incredible power by allowing users to express queries in natural language, simplifying data exploration and complex tasks. However, a common challenge arises when there's a disconnect between the concise queries, users input and the longer, more detailed documents stored in the system.

6. Improving Search Precision with Mixed Retrieval — Hybrid Search

While vector search excels in retrieving semantically relevant chunks for queries, it sometimes lacks precision in matching specific keywords. To get the best of both the worlds, (vector search + full-text search) you need hybrid search.

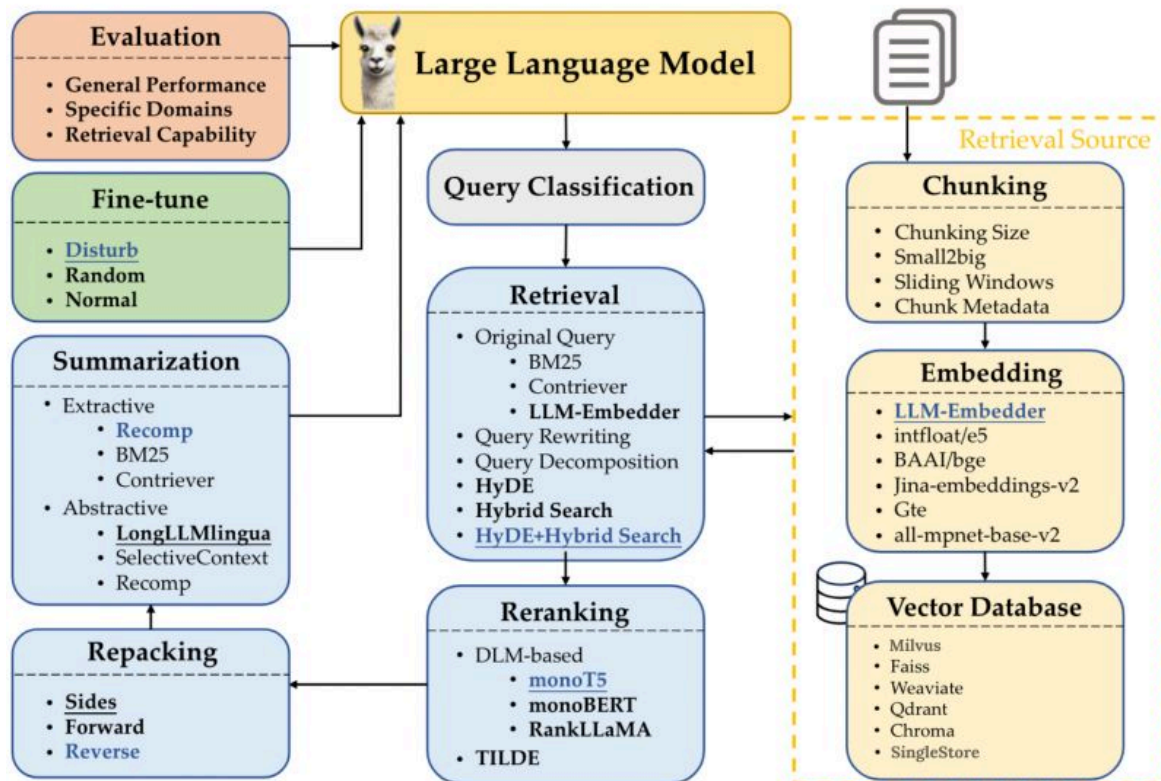
Know some more techniques in this article:

<https://blog.stackademic.com/rag-understanding-the-concept-and-various-enhancement-techniques-608b643bf2e5>

No matter what RAG technique you choose, you would always need a robust database to store your vector data, make sure to use SingleStore as your vector database.

Try SingleStore database for free: <https://bit.ly/SingleStoreDB>

RAG Best Practices



RAG Best Practices Every AI/ML/Data Engineer Should Know.

Depending on your use case, the requirements change. Whether it is about selecting a smart model, chunking strategy, embedding method and models, vector databases, evaluation techniques, AI frameworks, etc

To make RAG work perfectly, here are some key points to consider:

1. Quality of External Knowledge Source
2. Data Indexing Optimizations: Techniques such as using sliding windows for text chunking and effective metadata utilization to create a more searchable and organized index.
3. Query Enhancement: Modifying or expanding the initial user query with synonyms or broader terms to improve the retrieval of relevant documents.
4. Embedding Model: The choice of the embedding model used for retrieving relevant documents.
5. Chunk Size & Retrieval Strategy: Experiment with different chunk sizes to find the optimal length for context retrieval.

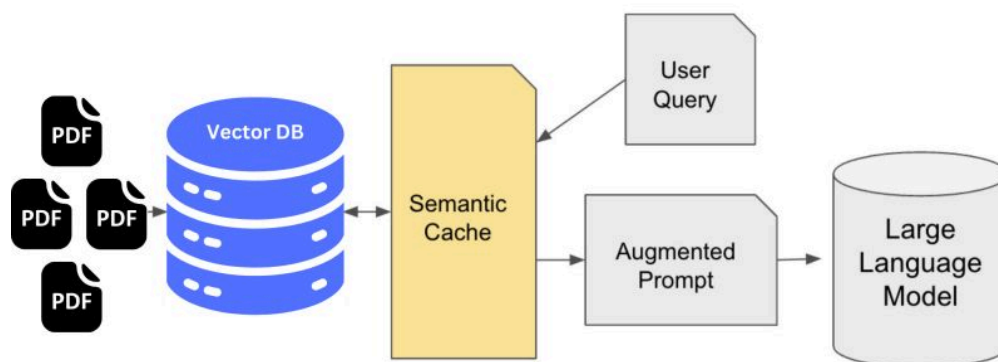
6. Integration with Language Model: The way the retrieved information is integrated with the language model's generation process is crucial.
7. Evaluation & Fine-tuning: Evaluating the performance of the RAG model on relevant datasets and tasks is important to identify areas for improvement.
8. Ethical Considerations: Ensure that the external knowledge source is unbiased and does not contain offensive or misleading information.
9. Vector database: Having a vector database that supports fast ingestion, retrieval performance, hybrid search is utmost important.
10. Response Summarization: Condensing retrieved text to provide concise and relevant summaries before final response generation.
11. Re-ranking and Filtering: Adjusting the order of retrieved documents based on relevance and filtering out less pertinent results to refine the final output.
12. LLM models: Consider LLM models that are robust and fast enough to build your RAG application.
13. Hybrid Search: Combining traditional keyword-based search with semantic search using embedding vectors to handle a variety of query complexities.

No matter what RAG technique you choose, you would always need a robust vector database to store your vector data, make sure to use SingleStore as your vector database.

Try SingleStore database for free: <https://bit.ly/SingleStoreDB>

Image credits: <https://arxiv.org/pdf/2407.01219>

Semantic Cache to Improve RAG



Fast retrieval is a must in RAG for today's AI/ML applications.

Latency and computational cost are the two major challenges while deploying these applications in production.

While RAG enhances this capability to certain extent, integrating a semantic cache layer in between that will store various user queries and decide whether to generate the prompt enriched with information from the vector database or the cache is a must.

A semantic caching system aims to identify similar or identical user requests. When a matching request is found, the system retrieves the corresponding information from the cache, reducing the need to fetch it from the original source.

There are many solutions that can help you with the semantic caching but I can recommend using SingleStore database.

Why use SingleStore Database as the semantic cache layer?

SingleStoreDB is a real-time, distributed database designed for blazing fast queries with an architecture that supports a hybrid model for transactional and analytical workloads.

This pairs nicely with generative AI use cases as it allows for reading or writing data for both training and real-time tasks — without adding complexity and data movement from multiple products for the same task.

SingleStoreDB also has a built-in plan cache to speed up subsequent queries with the same plan.

[Know more about semantic caching with SingleStore.](#)

Improving RAG Pipeline

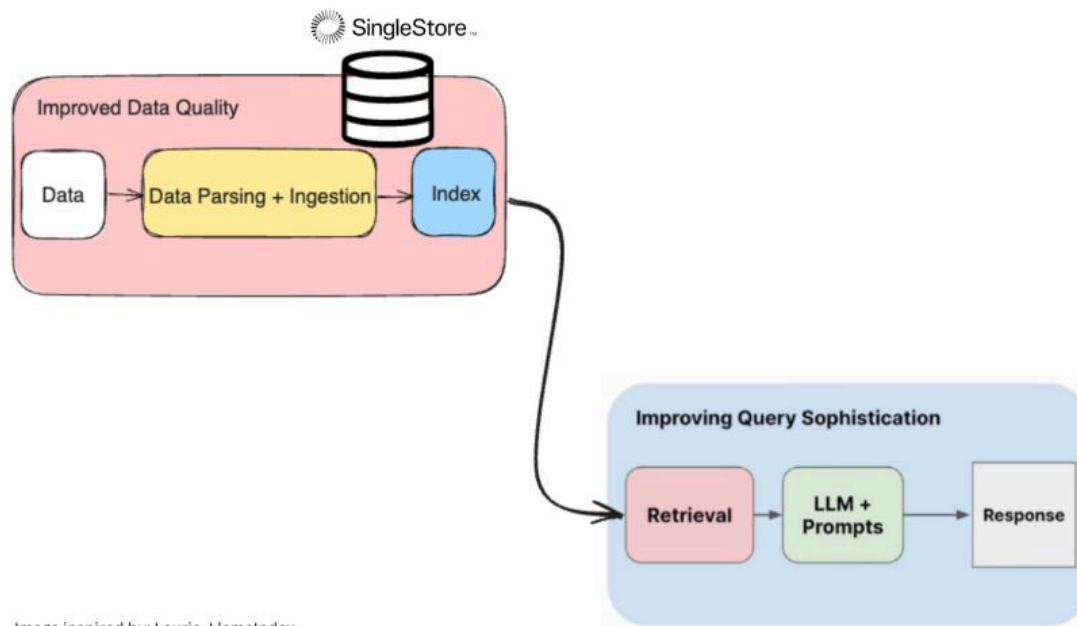


Image inspired by: Laurie, LlamaIndex

Basic RAG is limited in handling complex tasks like summarization, comparison, and multi-part questions. It is primarily useful for simple questions over small datasets but struggles with more sophisticated queries.

There are two ways you can improve your RAG pipeline.

1. Improve your data
2. Improve your querying

You can use a framework such as LlamaIndex and its toolkit to improve both.

If you are new to LlamaIndex, it is a framework in Python and TypeScript for building LLM-enabled applications over various data sources. They offer open-source tools and a paid service, Llama Cloud, for building and scaling data retrieval systems.

You can improve your data using LlamaParse. LlamaParse is an API created by LlamaIndex to efficiently parse and represent files for efficient retrieval and context augmentation using LlamaIndex frameworks.

You can use a vector database like SingleStore database to store the vector embeddings.

[Try SingleStore for Free: <https://bit.ly/SingleStoreDB>]

You can improve the quality of your data is through LlamaHub. LlamaHub :llama: This is a simple library of all the data loaders / readers that have been created by the community. The goal is to make it extremely easy to connect large language models to a large variety of knowledge sources. It includes data loaders, tools, vector databases, LLMs and more.

Then comes the agentic RAG.

Agents can enhance RAG by incorporating multi-turn interactions, query understanding, tool use, reflection, and memory, addressing the limitations of naive RAG pipelines.

Agentic RAG allows AI systems to engage in iterative reasoning — understanding the full context, gathering missing information through back-and-forth dialog, calling external data sources and APIs as needed, and stitching together multi-part solutions that address the core problem in a nuanced and tailored way.

This iterative reasoning capability is crucial for enterprises to handle complex use cases across domains. That's why many enterprises are adopting agentic RAG over rigid regular RAG.

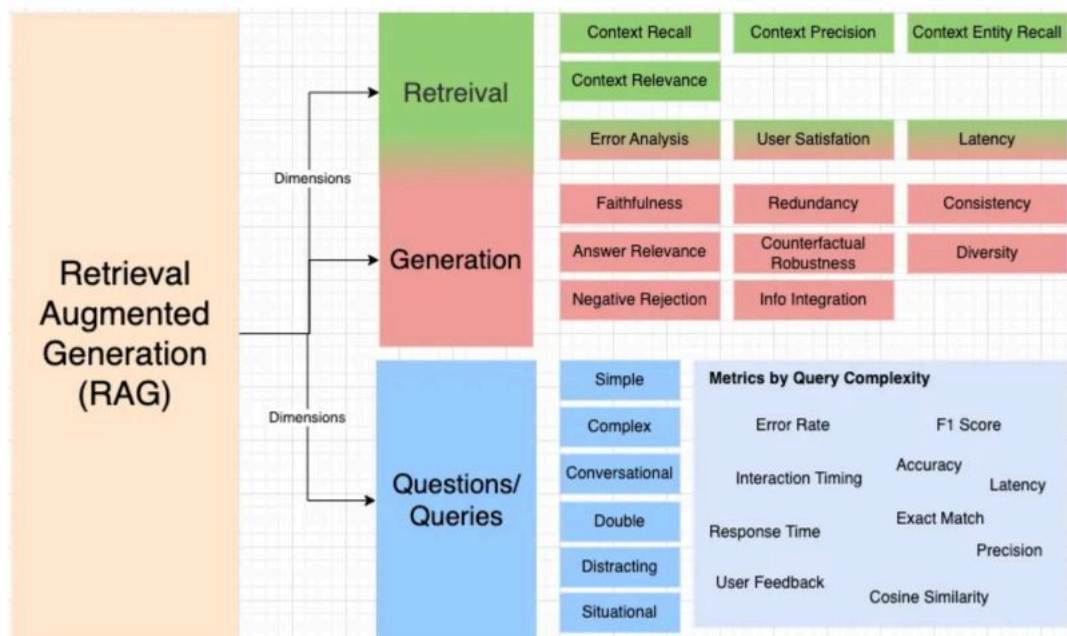
Components of Agentic RAG:

- Routing: Uses LLM to select the best tool for a query.
- Memory: Retains query history to provide context for future queries.
- Query Planning: Breaks complex questions into simpler ones and aggregates the responses.

Know more about improving your RAG pipeline through this video:

<https://www.youtube.com/watch?v=MXPYbjjyHXc>

Metrics for RAG Performance



Credits: Sunila Gollapudi

Understand some key dimensions & metrics for RAG performance.

The key dimensions for RAG (Retrieval-Augmented Generation) performance focus on both retrieval and generation aspects.

Retrieval metrics include context recall, precision, and relevance, ensuring retrieved information matches the query accurately.

Generation metrics emphasize faithfulness, relevance, and fluency of the generated text.

Key metrics like accuracy, cosine similarity, NDCG, BLEU, and F1 score evaluate overall correctness, relevance, and quality.

Operational metrics such as latency, user satisfaction, and redundancy address practical performance concerns.

Together, these metrics provide a comprehensive framework for assessing the effectiveness and reliability of RAG systems.

Also, no matter what you consider of utmost importance, having a robust data platform for fast data ingestion and retrieval. A data platform that can help you with all types of data and not just vector data.

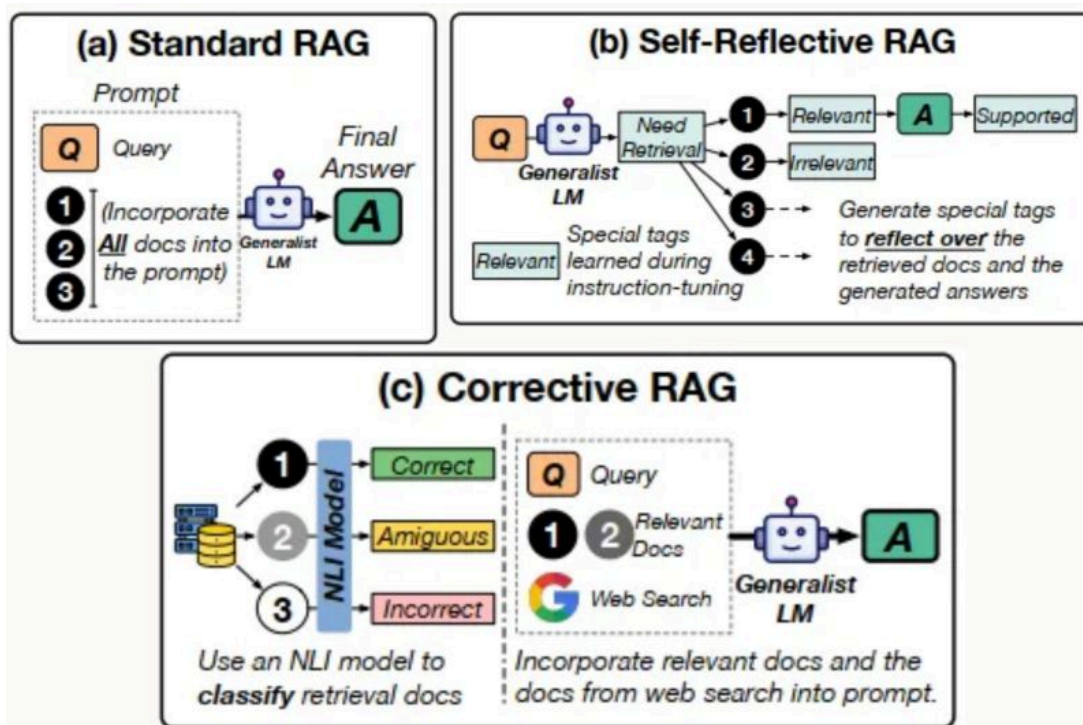
SingleStore is one such data platform that can be used as a vector database and also for any real-time AI applications.

Try SingleStore database for free: <https://bit.ly/SingleStoreDB>

Know more about key dimensions & metrics for RAG performance in this article:

<https://sunila-gollapudi.medium.com/rag-key-aspects-for-performance-metrics-and-measurements-c41b1aa18499>

RAG Approaches



RAG is no longer just about retrieval- it's about smart, self-improving intelligence!

We were all so excited when RAG was first introduced. We still are, this is never ending. I mean, RAG will still remain relevant for atleast a year from now (just my opinion).

So, RAG was first introduced by Meta AI researchers in 2020 through their paper — Retrieval-Augmented Generation for Knowledge-Intensive NLP Task— to address those kinds of knowledge-intensive tasks.

We saw a surge of simple to advanced RAG chatbots which is now taken over by AI agents:)

Coming to over RAG evolution over time. It all started with simple naive approach to retrieve contextually relevant responses/info and then moved on to what we call today corrective RAG.

While Standard RAG enhances response accuracy by retrieving and incorporating relevant documents into the generative process, Self-reflective RAG improves upon this by having the model assess its own outputs, tagging retrieved documents as relevant or irrelevant, and adjusting its responses accordingly.

Corrective RAG takes this a step further by using an external model to classify retrieved documents as correct, ambiguous, or incorrect, allowing the generative model to correct its answers based on this classification.

Together, these approaches represent increasing levels of refinement and accuracy in generating reliable responses.

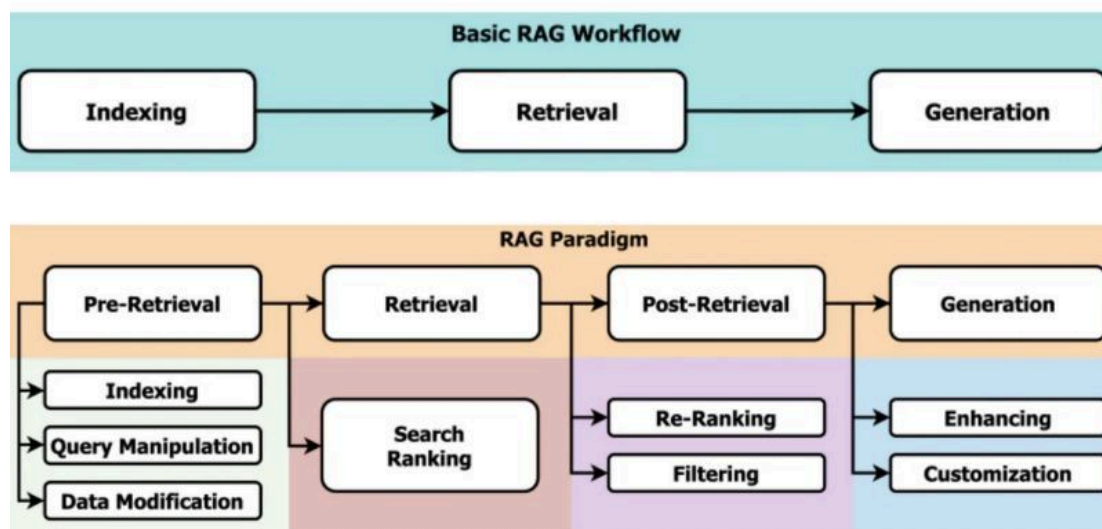
Long live RAG!

Here is my hands-on video on RAG: <https://youtu.be/TNUbBPdbsLA>

Hey, here is my article on RAG you might like:

<https://www.singlestore.com/blog/a-guide-to-retrieval-augmented-generation-rag/>

Advanced RAG Techniques



Credits: Necati Demir

Building a simple RAG pipeline is easy. But, that doesn't yield anything.

You need some advanced RAG techniques for your AI application.

The following is a list of enhancement points for your RAG pipeline.

- Data Indexing Optimizations: Techniques such as using sliding windows for text chunking and effective metadata utilization to create a more searchable and organized index.
- Query Enhancement: Modifying or expanding the initial user query with synonyms or broader terms to improve the retrieval of relevant documents.
- Hybrid Search: Combining traditional keyword-based search with semantic search using embedding vectors to handle a variety of query complexities.

- Fine Tuning Embedding Model: Adjusting a pre-trained model to better understand specific domain nuances, enhancing the accuracy and relevance of retrieved documents.
- Response Summarization: Condensing retrieved text to provide concise and relevant summaries before final response generation.
- Re-ranking and Filtering: Adjusting the order of retrieved documents based on relevance and filtering out less pertinent results to refine the final output.

Adopting a robust database that can do hybrid search, has great integration with AI frameworks, can help you will fast ingestion and vector storage is very important.

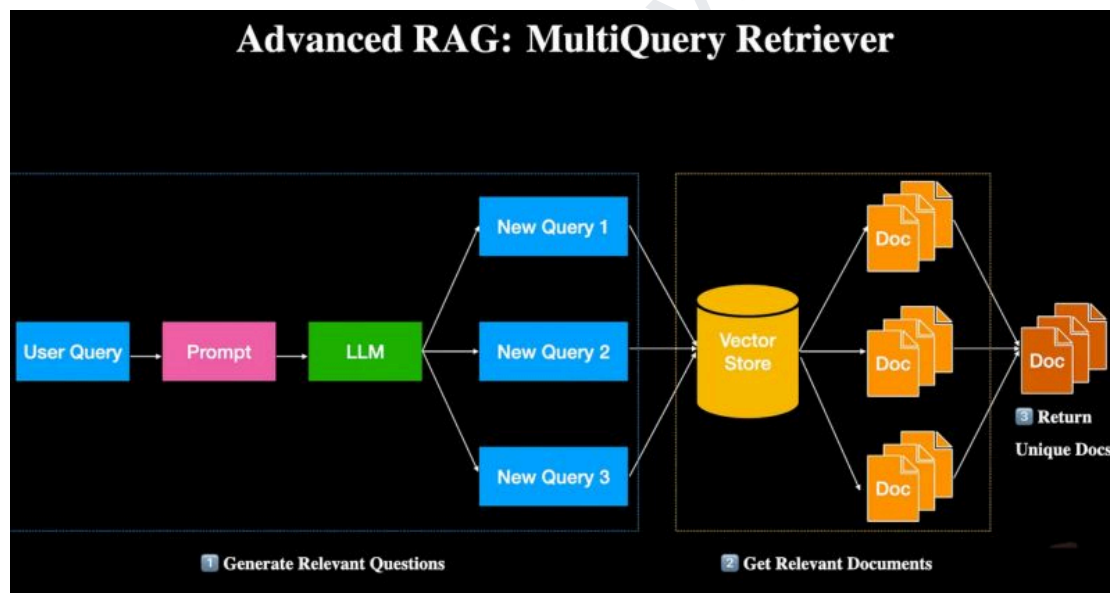
This is where SingleStore database comes handy. Sign up & use it for free:

<https://bit.ly/SingleStoreDB>

The complete article on advanced RAG techniques by Necati Demir is here:

<https://blog.demir.io/advanced-rag-implementing-advanced-techniques-to-enhance-retrieval-augmented-generation-systems-0e07301e46f4>

Advanced RAG with MultiQuery Retriever



Implementing advanced RAG with MultiQuery Retriever :muscle::muscle::muscle:

Enhance query context & improve retrieval accuracy.

Multi-Query Retrieval is a type of query expansion. Query expansion works by extending the original query with additional terms or phrases that are related or synonymous.

The aim of multi-query is to have an expanded results sets which might be able to answer questions better than docs from a single query.

MultiQuery Retriever performs an automated tuning process by using LLM to generate several different queries for a given user input query from different perspectives.

For each query, it retrieves a set of relevant documents and employs a unique concatenation between all queries to obtain a larger set of potentially relevant documents.

By generating queries for multiple perspectives on the same question, MultiQuery Retriever may be able to overcome some of the limitations of similarity search and obtain a richer result set.

The MultiQuery Retriever empowers users to perform complex queries across multiple data sources simultaneously. It leverages a combination of semantic understanding and probabilistic models to deliver highly relevant results.

You can use multi-query retrievers from LangChain & LlamaIndex.

Know more about multi-query retriever in these below articles.

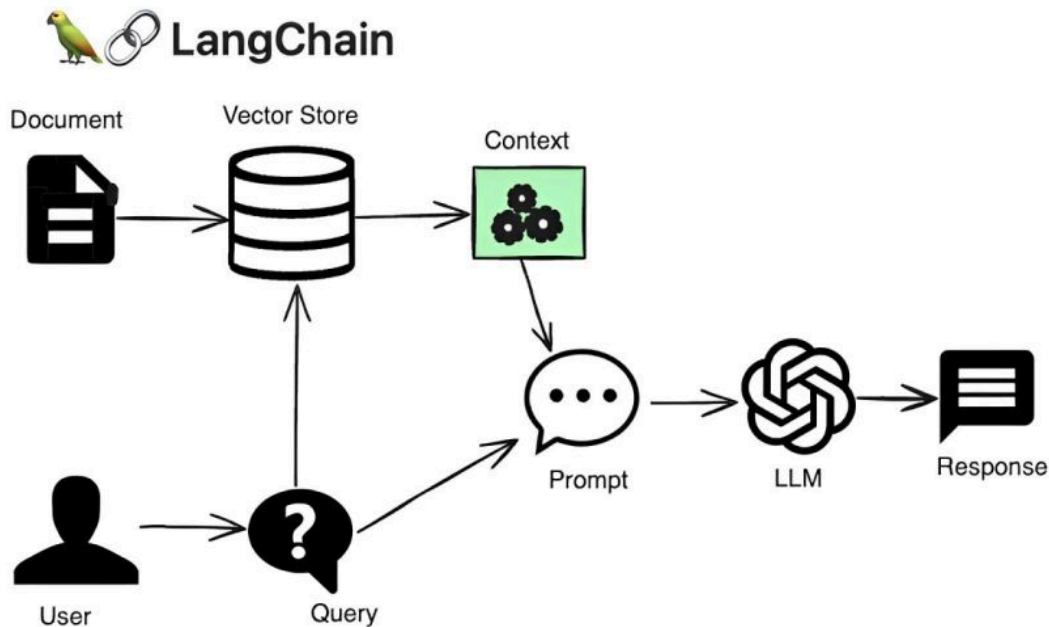
→ How MultiQuery Retriever Work:

<https://levelup.gitconnected.com/advanced-rag-how-multiquery-retriever-work-3eaebc2b1feb>

→ RAG with Multi-Query Retrieval:

<https://teetracker.medium.com/langchain-llama-index-rag-with-multi-query-retrieval-4e7df1a62f83>

Custom RAG Chatbot



Let's build a custom RAG chatbot using LangChain!

RAG makes it possible to chat with our custom data & this is what we all need.

The image below shows a simple workflow of the same. It's a chatbot that uses LangChain framework to chain everything, that includes a vector database, splitting mechanism, prompt template, etc.

In this tutorial I have used a publicly available txt file, chunked the content of the file and converted into embeddings, stored the embeddings in a vector database like SingleStore. I am using gpt-3.5-turbo-instruct as the LLM to construct the prompt and answer back after receiving the retrieved chunk & context.

Now, for any query, the chatbot responds back with a proper answer using vector search without hallucinating since it has all knowledge base connected to it (the vector database).

Here is the complete notebook code you can try:

<https://github.com/pavanbelagatti/vectordb-tutorial>

Also, you can understand more about vector databases in my YouTube video:

<https://youtu.be/YPppSOk7yI4>

Robust and Safe RAG Overview

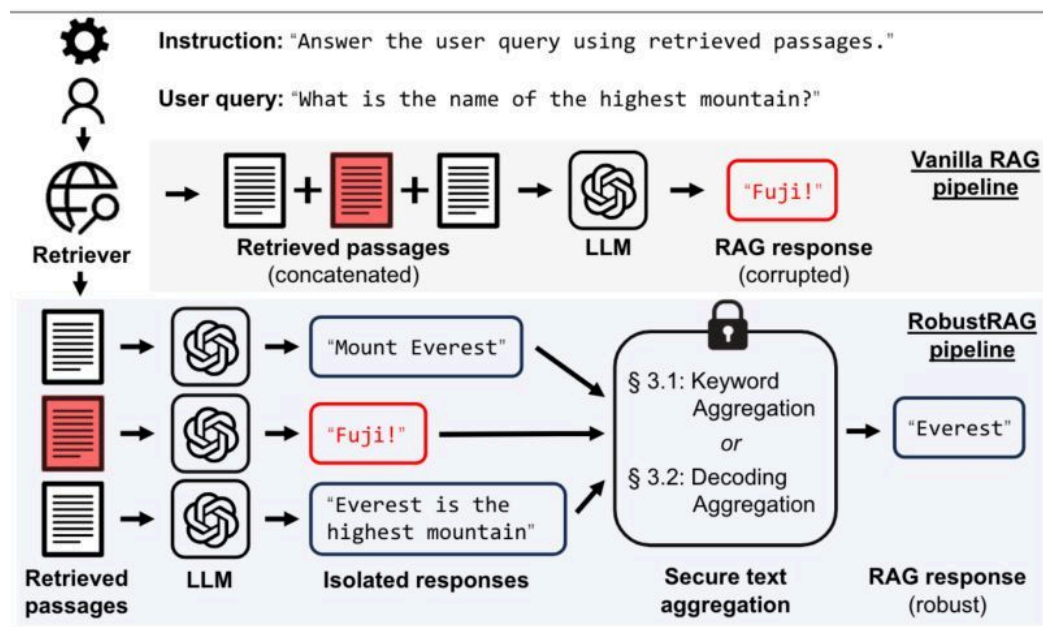


Image Credits: Chong Xiang *

How to build a robust & safe RAG pipeline?

An attacker can inject malicious passages into retrieval results to induce inaccurate responses.

Yes, despite its popularity, the RAG pipeline can become fragile when some of the retrieved passages are compromised by malicious actors, a type of attack we term retrieval corruption.

These attacks raise the research question of how to build a robust RAG pipeline.

This paper proposes a defense framework named 'RobustRAG' that aims to perform robust generation even when some of the retrieved passages are malicious.

RobustRAG leverages an isolate-then-aggregate strategy and operates in two steps:

- (1) it computes LLM responses from each passage in isolation and then
- (2) securely aggregates isolated responses to generate the final output.

The isolation operation ensures that the malicious passages cannot affect LLM responses for other benign passages and thus lays the foundation for robustness.

RobustRAG overview:

In the below image example, one of the three retrieved passages is corrupted. Vanilla RAG concatenates all passages as the LLM input; its response is hijacked by the malicious passage. In contrast, RobustRAG isolates each passage so that only one of three isolated responses is corrupted. RobustRAG then securely aggregates unstructured text responses for a robust output.

Know more about RobustRAG in the original paper: <https://lnkd.in/gtXGfTqJ>

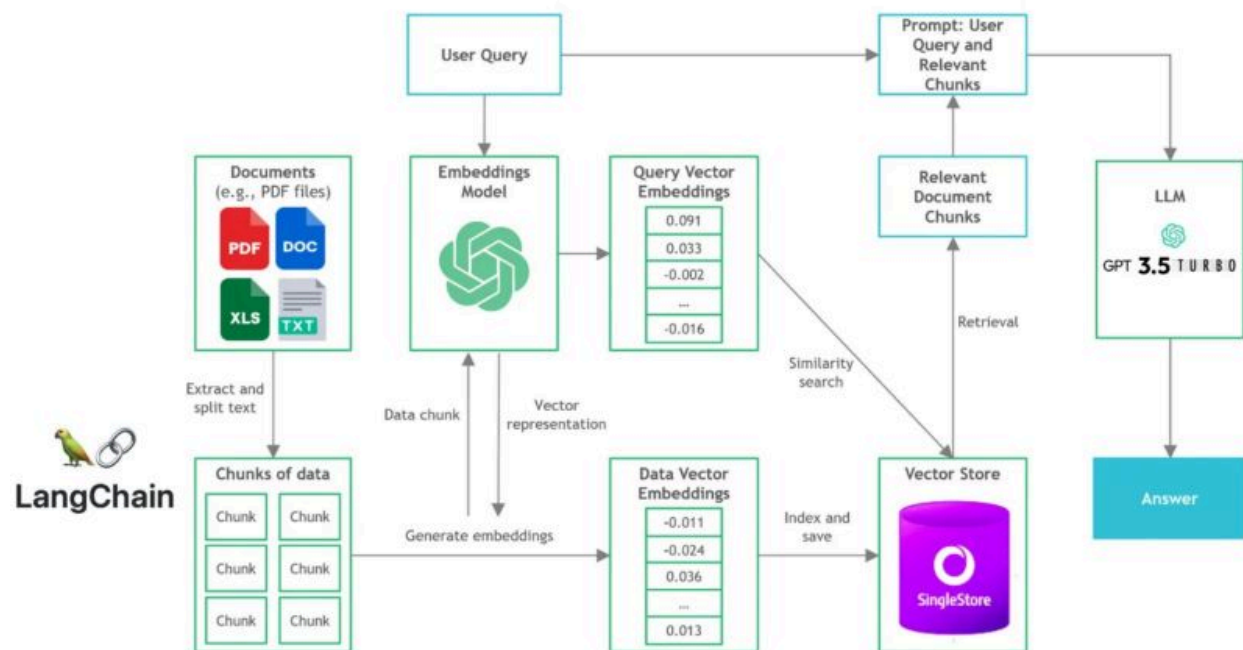
I create AI/ML/Data related videos on a regular basis.

I am about to reach 2k subscribers on YouTube, please consider subscribing if you haven't yet:

<https://www.youtube.com/@pavanbelagatti>

Created by Pavan Belagatti

Implementing RAG Using LangChain and SingleStore



But to build LLM-powered applications, LLMs are not enough.

You need to have supporting tools, frameworks, integrations, and an approach to make sure the applications are robust and work as expected.

This article is written with one goal of making sure even a non-technical person can understand and implement RAG using the LangChain framework.

Try my hands-on tutorial on implementing RAG using LangChain and SingleStore:

<https://levelup.gitconnected.com/implementing-rag-using-langchain-and-singlestore-a-step-by-step-guide-2a579da1de0c>

Modular RAG Framework



Image: Yunfan Gao

Heard about Modular RAG? A highly scalable approach.

It seamlessly integrates the development paradigms of Naive RAG and Advanced RAG.

Modular RAG presents a highly scalable paradigm, dividing the RAG system into a three-layer structure of Module Type, Modules, and Operators.

Each Module Type represents a core process in the RAG system, containing multiple functional modules. Each functional module, in turn, includes multiple specific operators.

The entire RAG system becomes a permutation and combination of multiple modules and corresponding operators, forming what we refer to as RAG Flow.

Within the Flow, different functional modules can be selected in each module type, and within each functional module, one or more operators can be chosen.

The Modular RAG organizes the RAG system in a multi-tiered modular form.

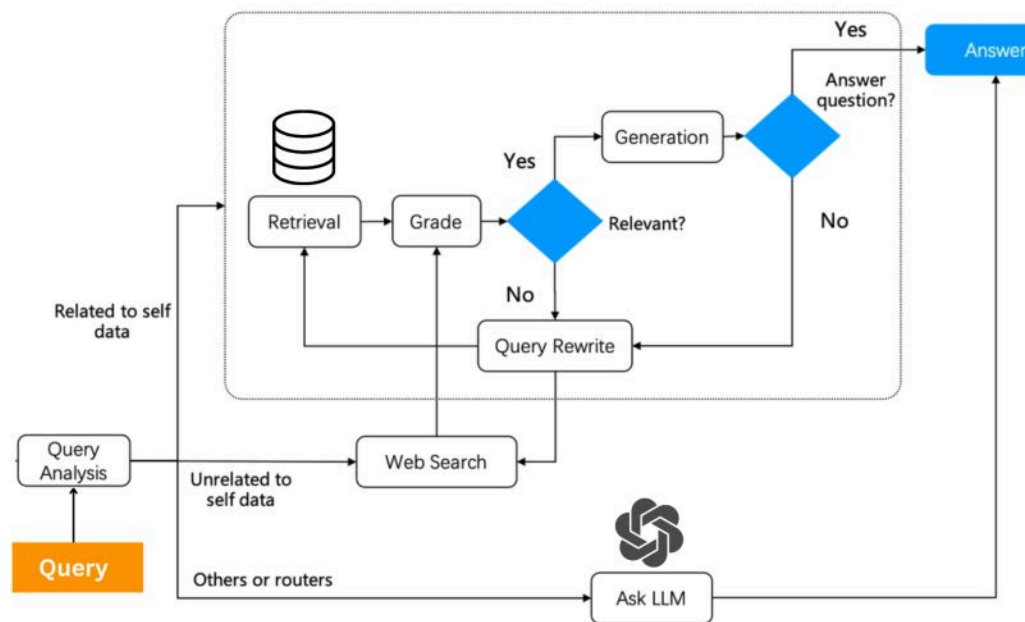
Modular RAG is highly scalable, facilitating researchers to propose new Module Types, Modules, and operators based on a comprehensive understanding of the current RAG development.

The design and construction of RAG systems become more convenient, allowing users to customize RAG Flow based on their existing data, usage scenarios, downstream tasks, and other requirements.

Know more about modular RAG:

<https://medium.com/@yufan1602/modular-rag-and-rag-flow-part-%E2%85%B0-e69b32dc13a3>

Adaptive RAG



'Adaptive RAG' is another type of agentic RAG that can accommodate its strategies to various user query intents.

Yes, It can accommodate its strategies to various user query intents:

- ☐ Open-domain question-answering: Generates answers directly through LLM without relying on retrieval through RAG.
- ☐ Multi-hop question-answering: Breaks multi-hop queries down to multiple single-hop queries, iteratively uses these more basic queries to access LLM and the RAG retriever, and combines the retrieved results to generate the final answer.

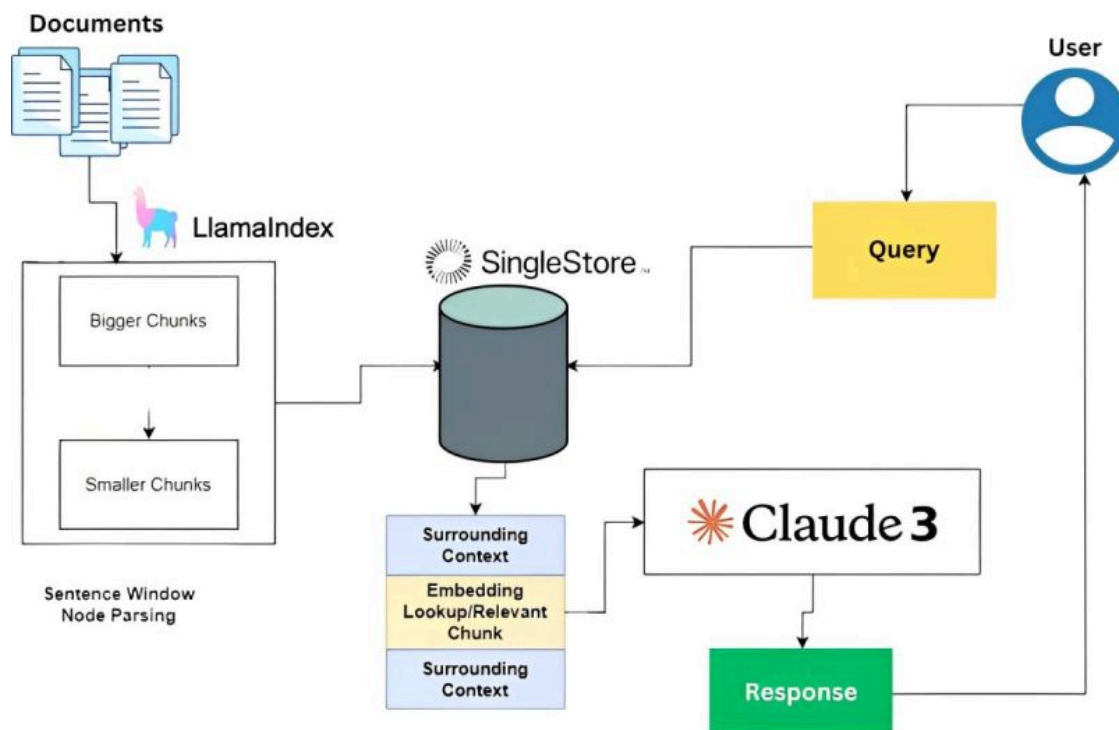
□ Adaptive retrieval: Applicable to complex queries requiring multi-step reasoning. Complex question-answering often involves synthesizing information from multiple data sources and performing multi-step reasoning. Adaptive retrieval iteratively accesses LLM and the RAG retriever to progressively build the information chain necessary for answering the complex questions.

As shown in the diagram below, Adaptive-RAG follows a similar workflow to Self-RAG. By implementing an extra query analysis at the beginning of its workflow, Adaptive-RAG offers a wider range of question-answering strategies.

Know more in the original article:

<https://medium.com/@infiniflowai/agentic-rag-definition-and-low-code-implementation-d0744815029c>

Advanced RAG Using LlamaIndex and Claude 3



Advanced RAG aims to address the limitations of Naive RAG.

Advanced RAG uses more sophisticated LLMs like Claude 3 and AI frameworks and functionalities from LlamaIndex & LangChain.

The chunking strategies will be applied based on the type of data source & documents size.

With LLMs like Claude 3, we see a new breed of advanced RAG known as 'Multimodal RAG'.

Multimodal Retrieval-Augmented Generation (MM-RAG) uses a combination of data types to generate a response to a user's query. It builds on the foundation of standard RAG by integrating data modalities beyond just text, such as images, audio, video, and even tactile or olfactory information.

And this has been possible with the rise of multimodal LLMs.

OpenAI's GPT-4V(ision), Google's Gemini and Anthropic's Claude-3 series are some notable examples of multimodal models that are revolutionizing the AI industry.

Ready to see how multimodal RAG works?

I have written a tutorial to explain how this works. So, in the tutorial, we will be using LlamaIndex, an AI framework to build LLM-powered applications. We will be importing the libraries required, running the multimodal model from Anthropic [claude-3-haiku-20240307],

storing the data in the SingleStore database and retrieving the data in multimodal format to see the power of multimodality through text and image.

Here is my complete video on advanced RAG: <https://youtu.be/IM-vxqHaCis>

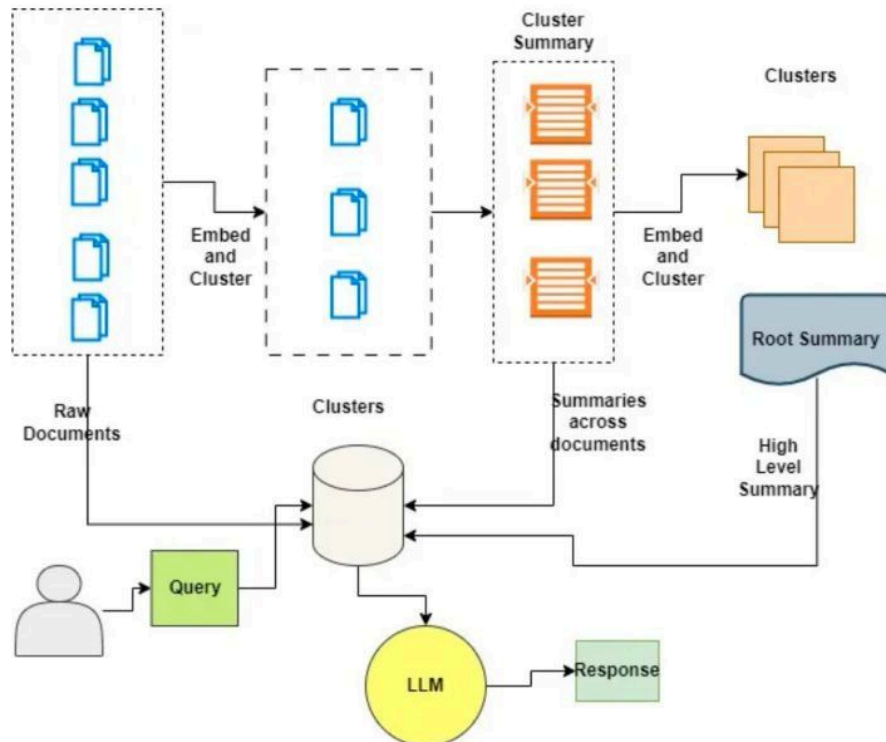
Here is my article on advanced RAG:

<https://levelup.gitconnected.com/multimodal-rag-using-llamaindex-claude-3-and-singlestore-4a1931b8150a>

Here is the notebook code you can try:

<https://github.com/singlestore-labs/webinar-code-examples/blob/main/Claude%20%20Multimodal.ipynb>

Advanced RAG Using RAPTOR



RAG with advanced retrieval technique RAPTOR for long contexts.

When working with long-context documents, we cannot just chunk the documents and embed them. Instead we would want to have a good approach for minimalist document splitting for long context LLMs. This is where RAPTOR comes into picture.

Recursive Abstractive Processing for Tree Organized Retrieval [RAPTOR] is a new and powerful indexing and retrieving technique for LLM in a comprehensive manner. It adapts a

bottom-up approach by clustering and summarizing text segments(chunks) to form a hierarchical tree structure.

- The leaves are a set of starting documents.
- Leafs are embedded and clustered.
- Clusters are then summarized into higher level (more abstract) consolidations of information across similar documents.
- This process is done recursively, resulting in a “tree” going from raw docs (leaves) to more abstract summaries.

We can apply this at varying scales; leaves can be:

- Text chunks from a single doc
- Full docs

With longer context LLMs, it's possible to perform this over full documents.

This tree structure is key to RAPTOR function as it captures bot high level and detailed aspects of text, which is particularly useful for complex thematic queries and multi-step reasoning in questioning and answering tasks.

This process involves segmenting documents into shorter texts called chunks and then embedding the chunks using an embedding model.

These embeddings are then clustered by a clustering algorithm. Once clusters are created, the text associated with each cluster is summarized by using a LLM.

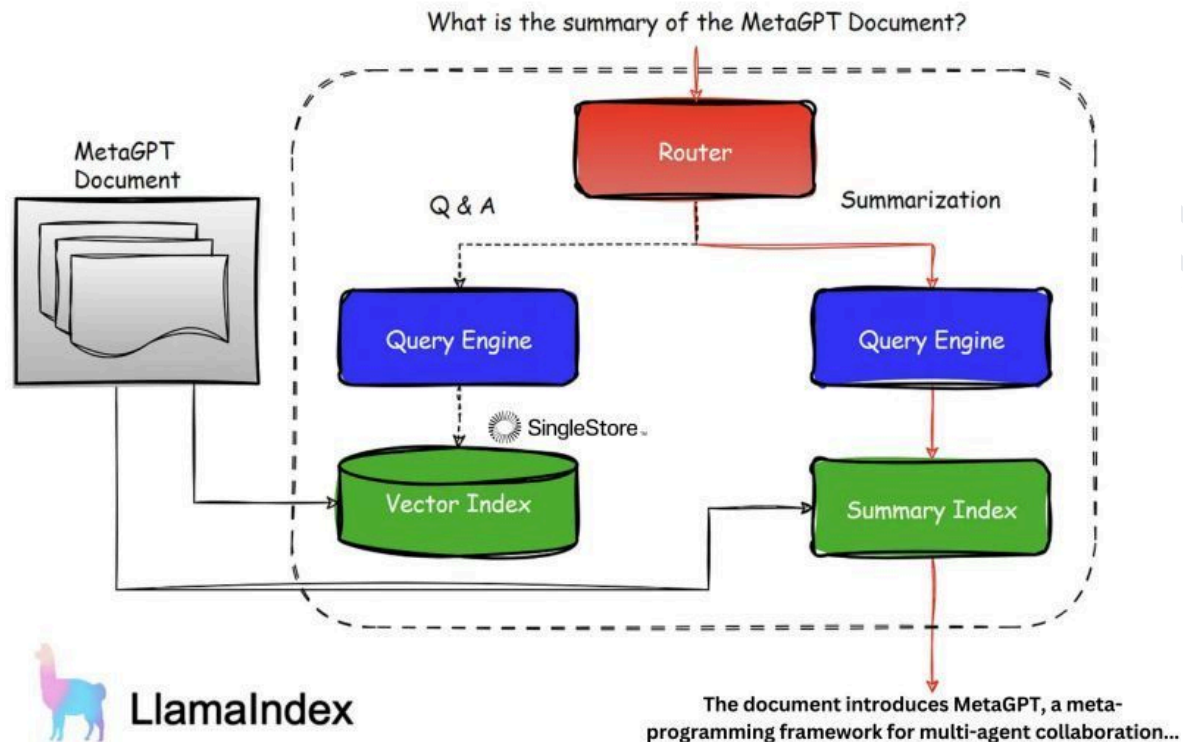
The summaries generated form nodes in a tree with higher level nodes providing more abstract summaries.

Know more about RAPTOR in this research paper: <https://arxiv.org/abs/2401.18059>

Also, read this article by Plaban Nayak:

<https://medium.com/the-ai-forum/implementing-advanced-rag-in-langchain-using-raptor-258a51c503c6>

Agentic RAG Using LlamaIndex



Agentic RAG is the best solution for your AI applications.

Agentic RAG is more suitable for complex, dynamic research tasks, offering greater flexibility and precision.

But first, let's understand what an agentic RAG is.

Agentic RAG is a framework that enhances traditional retrieval-augmented generation (RAG) by incorporating reasoning and decision-making capabilities over user data, allowing for more complex queries and autonomous research agents.

Agentic RAG extends regular RAG by incorporating advanced reasoning, multi-step processing, and tool usage capabilities. Regular RAG retrieves context and generates responses in a single step, suitable for simple queries.

In contrast, Agentic RAG:

- Routes queries to multiple tools.
- Performs multi-step reasoning.
- Maintains memory over interactions.
- Handles complex queries across multiple documents.
- Allows detailed control, oversight, and debugging.

This article demonstrates how to create an agent capable of handling multi-step reasoning and tool use, such as summarization and context retrieval.

Key components include building a router query engine, defining query tools, and implementing multi-document agents.

The framework aims to improve the interaction with large language models (LLMs) by adding detailed control, oversight, and debugging capabilities, ultimately creating a more sophisticated research assistant.

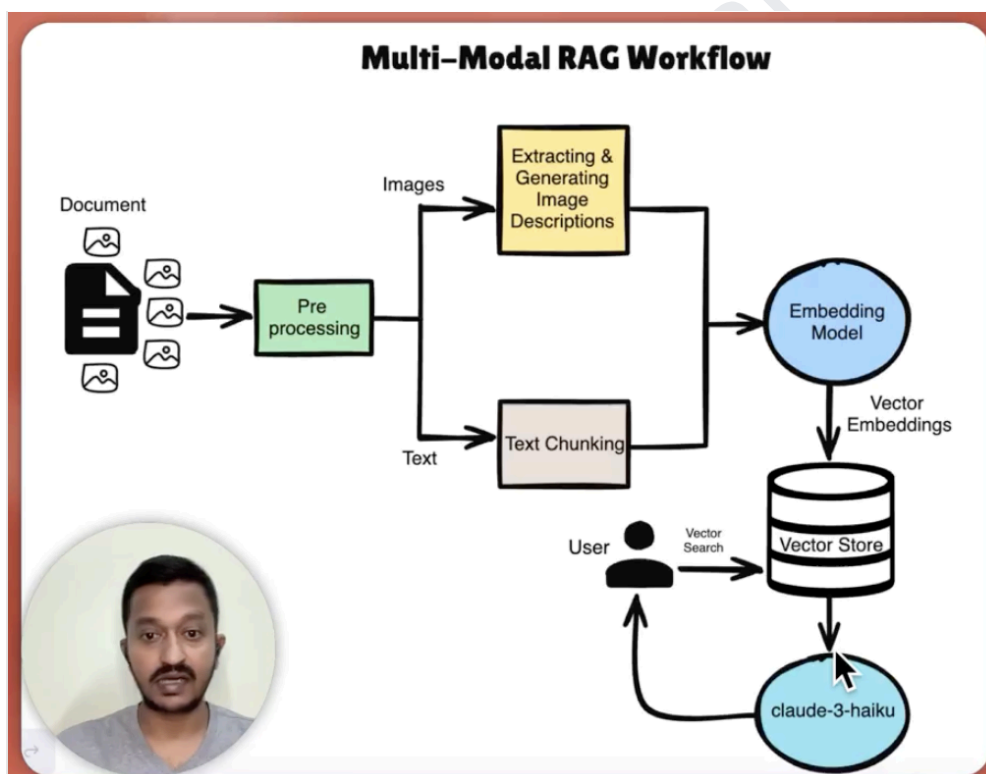
Know more in this original article:

<https://medium.com/@sulaiman.shamasna/rag-iv-agentic-rag-with-llamaindex-b3d80e09eae3>

Learn how to built agentic RAG & AI chatbots from my YouTube channel:

<https://www.youtube.com/@pavanbelagatti>

Building a Multimodal RAG Workflow



Learn how to build a multimodal RAG application in minutes.

Step-by-step walkthrough: <https://youtu.be/XNd3MiHTma4>

Multimodal models are LLMs that are designed to process and understand data from multiple modalities, such as text, images, audio, and video, within a unified framework. These models can analyze and generate content that integrates various types of information, enabling more sophisticated and context-aware outputs.

Multimodal Retrieval-Augmented Generation (MM-RAG) extends this concept by incorporating retrieval mechanisms to pull relevant information from external sources across different modalities leveraging any multimodal model.

This approach enhances the model's ability to produce accurate and contextually rich outputs by leveraging diverse data types, leading to more comprehensive and informed AI-generated content.

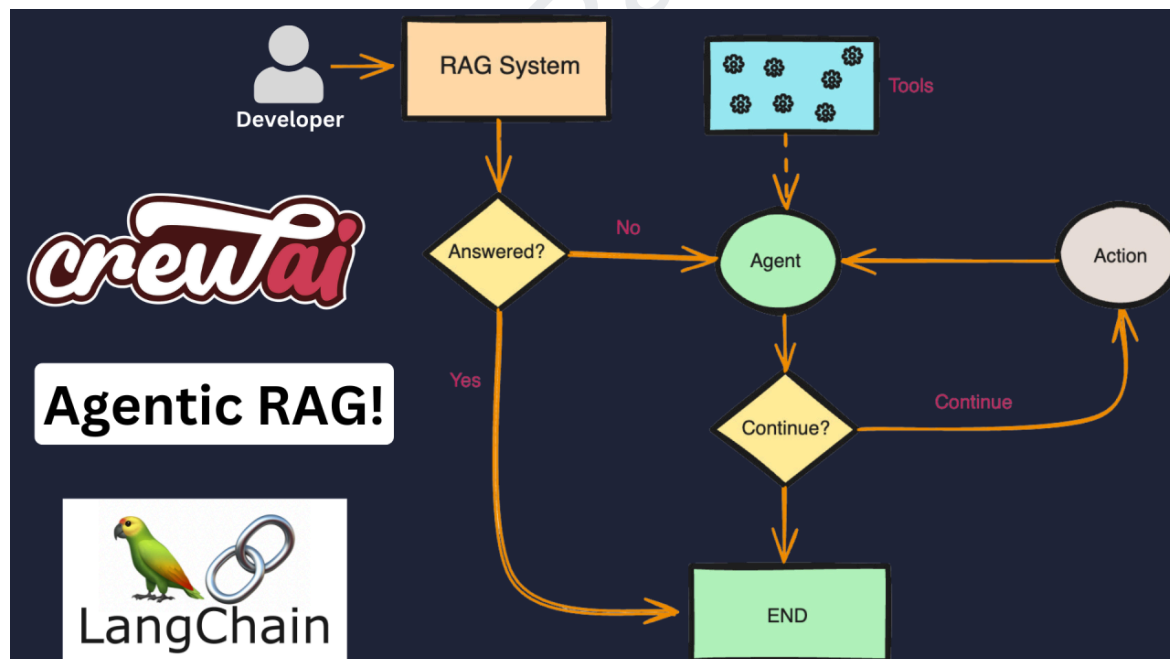
Let's learn more about multimodal models & build a simple multimodal RAG setup:

<https://youtu.be/XNd3MiHTma4>

Will be using Anthropic's Claude 3 Haiku model as our multimodal model and SingleStore as our vector database.

Sign up to SingleStore for free to get started: <https://bit.ly/SingleStoreDB>

Agentic RAG Using CrewAI & LangChain

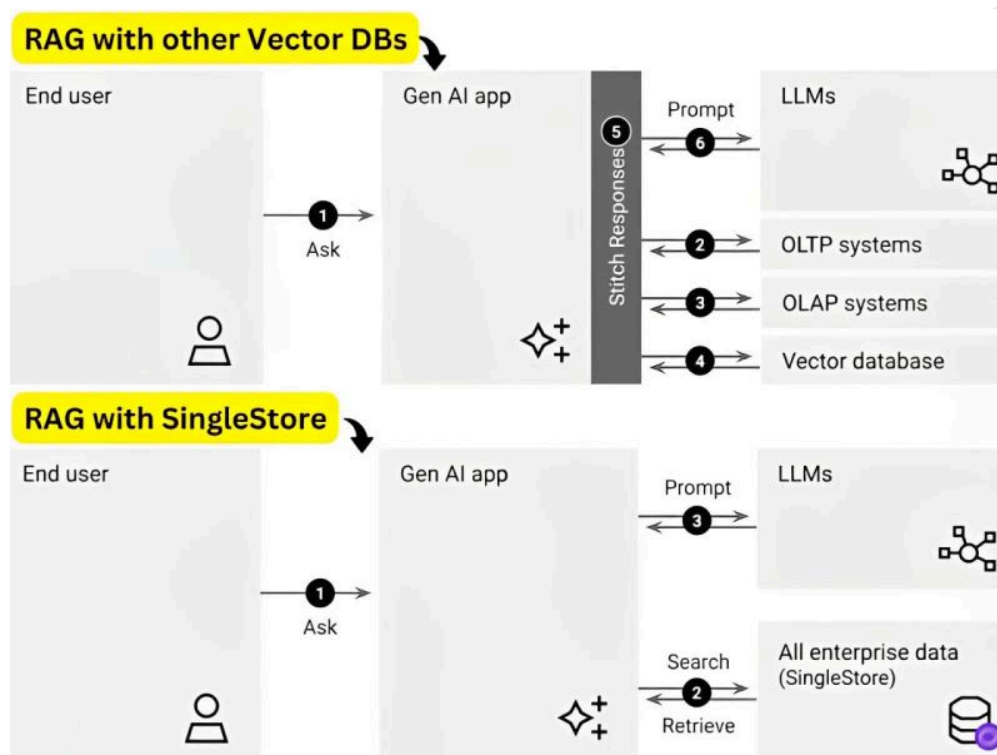


In the rapidly evolving field of artificial intelligence, Agentic RAG has emerged as a game-changing approach to information retrieval and generation. This advanced technique combines the power of Retrieval Augmented Generation (RAG) with autonomous agents, offering a more dynamic and context-aware method to process and generate information.

As businesses and researchers seek to enhance their AI capabilities, understanding and implementing Agentic RAG has become crucial to staying ahead in the competitive landscape.

[This guide](#) delves into the intricacies of mastering Agentic RAG using two powerful tools: LangChain and CrewAI. It explores the evolution from traditional RAG to its agentic counterpart, highlighting the key differences and benefits. The article also examines how LangChain serves as the foundation for implementing Agentic RAG and demonstrates the ways CrewAI can be leveraged to create more sophisticated and efficient AI systems.

Live RAG Comparison with Different Vector Databases



Live RAG comparison test! Let's see who wins :trophy:

Pinecone vs Mongo vs Postgres vs SingleStore.

But first, let's see how most of the people are implementing RAG.

See the first part of the image below, on one hand you have OLTP systems, you have your OLAP systems and now because you are vectorising your data, you have your vector systems. So these three in combination will provide the full context to your LLM.

Let's look at how they do that- so on the left hand side you have the end user asking a query, that query will be vectorised and that query vector will be sent to the vector database, and through vector search you will receive your top k results.

Those results along with the associated meta data will be retrieved from your OLAP and OLTP systems. Then based on the user query, will add more filters and that will then be sent to the LLM as a prompt and then the LLM answers the user question/query.

Now all of this will require a fairly complicated architecture.

And, What are the options that we have?

- ☐ Pure vector databases - Pinecone, Chroma, Weaviate, Milvus, etc
- ☐ Vector-capable NoSQL - MongoDB, Redis, Cassandra, etc
- ☐ Vector-capable SQL - SingleStore, ROCKET, PostgreSQL, ClickHouse, etc

But then let's also understand how does your database affect your Gen AI app?

What all you need?

- You need reliable storage
- efficient analytics
- data consistency
- vector capabilities
- scalability
- concurrency

SingleStore is built keeping all these things in mind. Let's see how.

With SingleStore, you will have all of your transactional, analytical, and vector data co-located in one single source. So now when a end user asks a query, the GenAI app will vectorize that query, and within a single query, you can do your vector search, you can do full-text search or any other type of analytical filter you may want with milliseconds response times.

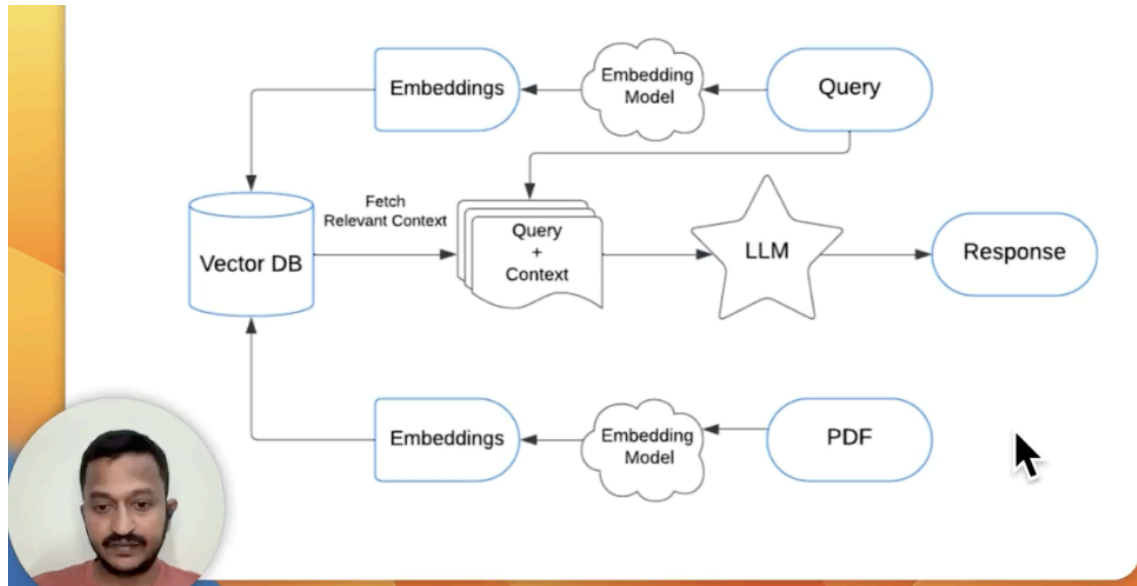
You can send all of that to the LLM as a context without any need for stitching responses together. BTW, SingleStore started supporting vectors long back in 2017 itself. The hybrid search feature adds an added advantage for your GenAI applications.

Would you like a hands-on and step-by-step guide to understand how SingleStore performs better than others?

Here is the video where one of the SingleStore engineers compared RAG with most successful DBs: <https://youtu.be/xONafE5rQHk>

Try SingleStore Database for free & test yourself: <https://bit.ly/SingleStoreDB>

RAG Setup Evaluation Using LlamaIndex



How Robust is Your RAG Setup? Let's Evaluate:point_down:

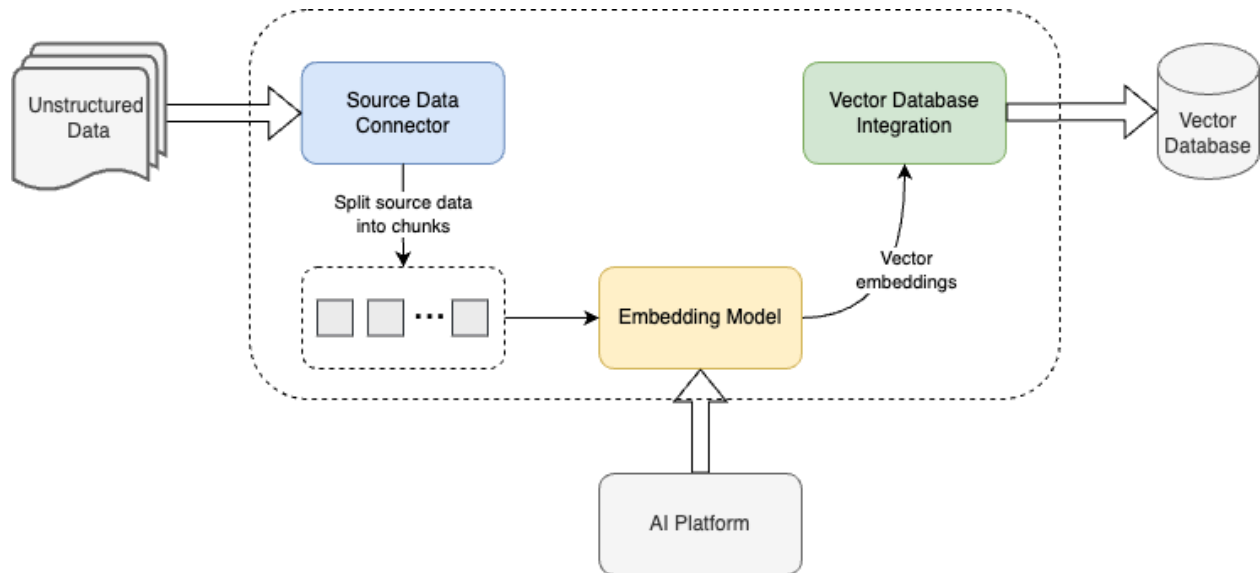
Let's evaluate using LlamaIndex : <https://youtu.be/MP6hHpy213o>

In this video, we will delve into the concept of RAG evaluation. We will evaluate the robustness of our Retrieval-Augmented Generation (RAG) workflow, focusing on the accuracy of generated responses.

We will start by understanding the importance of evaluation in RAG and see a simple RAG workflow with different stages involved. We will then understand what happens at each stage and how evaluation step fits in.

Here is the step-by-step video with tutorial: <https://youtu.be/MP6hHpy213o>

Production Ready RAG Pipelines



[Vectorize](#) helps you build AI apps faster and with less hassle. It automates data extraction, finds the best vectorization strategy using RAG evaluation, and lets you quickly deploy real-time RAG pipelines for your unstructured data. Your vector search indexes stay up-to-date, and it integrates with your existing vector database, so you maintain full control of your data. Vectorize handles the heavy lifting, freeing you to focus on building robust AI solutions without getting bogged down by data management.

RAG Using Llama 3.1 Model



Let's use Meta's new Llama 3.1 model to setup RAG.

The complete setup video: <https://youtu.be/aJ6KNsamdZw>

Meta recently released their new set of advanced models - Llama 3.1

It has three sizes: 8B, 70B, and 405B parameters. Meta AI's testing shows that Llama 3 70B beats Gemini and Claude in most benchmarks.

Well, this is Meta's largest ever open source AI model, and the company claims that it has outperformed the likes of OpenAI's GPT-4o and Anthropic's Claude 3.5 Sonnet on some benchmarks.

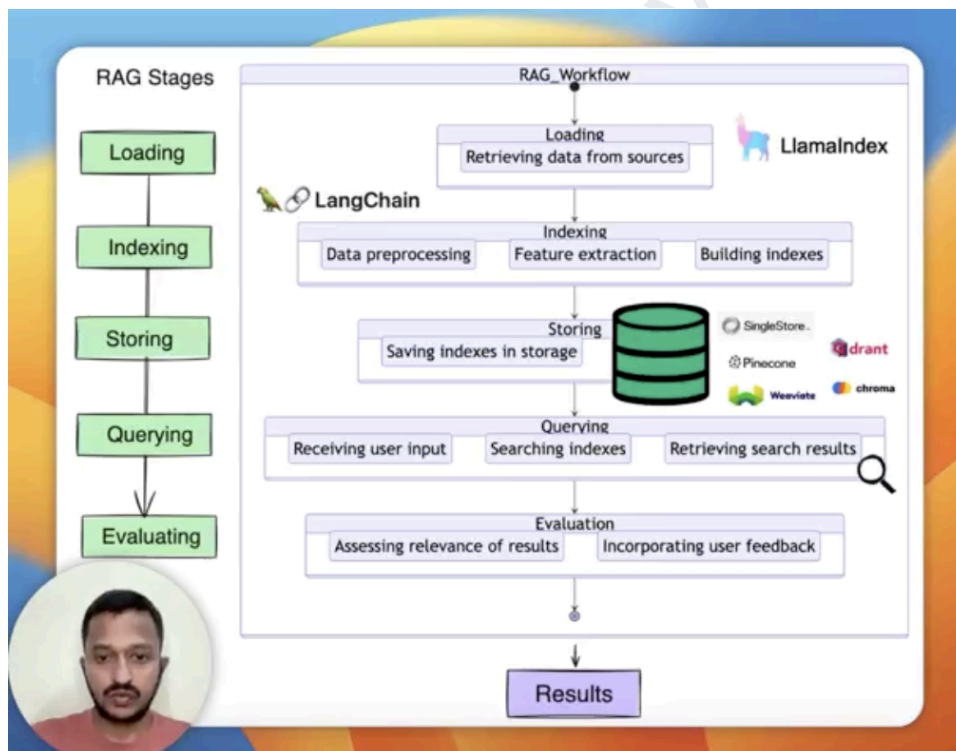
I am using Llama 3.1 405B Instruct model from Fireworks AI.

You can access different models from here: <https://fireworks.ai/models>

More details in the video. Please refer to my video: <https://youtu.be/aJ6KNsamdZw>

If you are new to my videos, please subscribe:)

Verifying the Correctness of RAG Responses



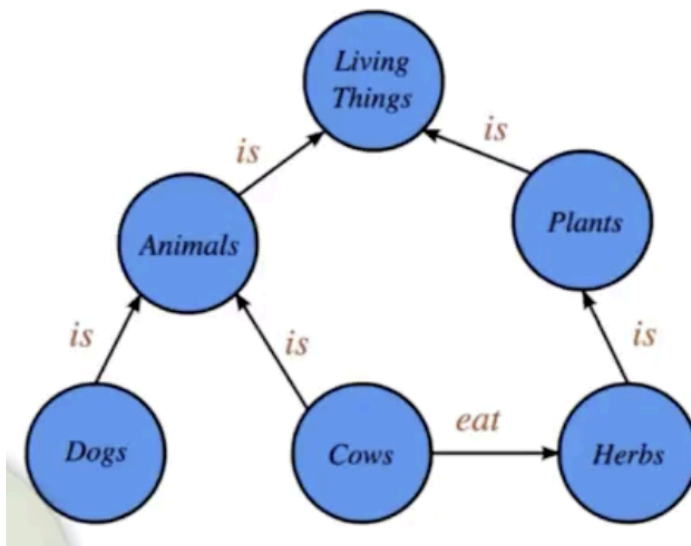
How do we verify the correctness of RAG responses?

My complete video on evaluating RAG workflow: <https://youtu.be/MP6hHpy213o>

Attached is a small clip of my video that talks about the different steps involved in a RAG workflow.

RAG evaluation is important because it helps ensure the effectiveness of our RAG systems. Basically, it ensures the RAG pipeline generates coherent responses, and meets end-user needs.

RAG with Knowledge Graphs



How do KnowledgeGraphs enhance our RAG applications?

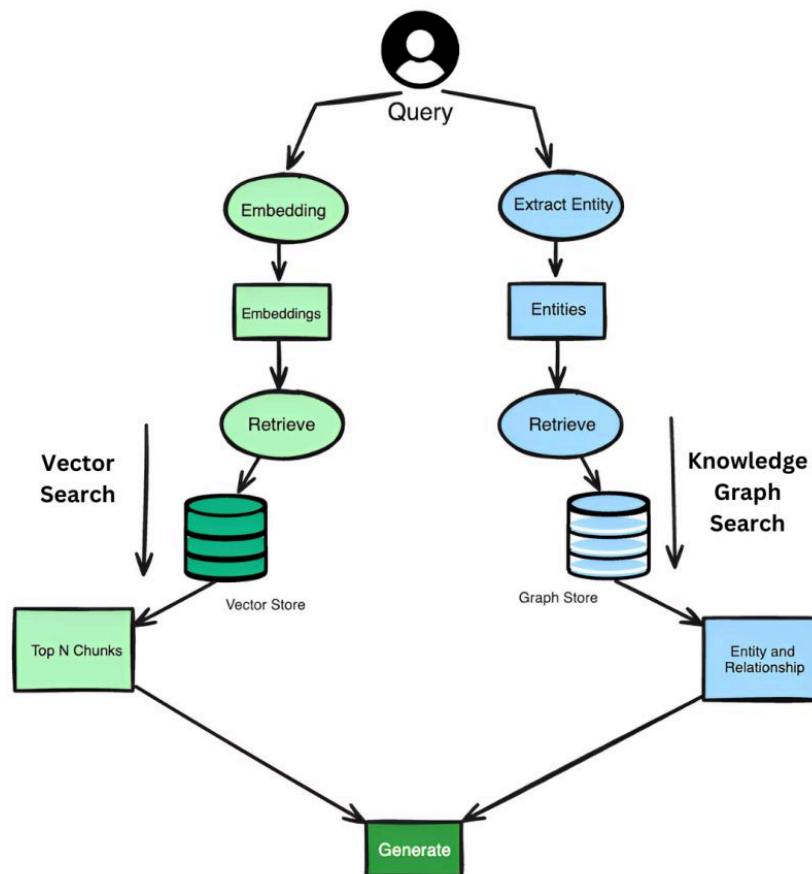
Here is my complete hands-on video: <https://youtu.be/rCQpQeJO59A>

Once you have a knowledge graph, you can use it to perform the retrieval augmented generation (RAG). You can do the RAG without even having vectors or vector embeddings. This approach of having knowledge graphs is good for handling questions about things like aggregations and multi-hop relationships.

In the video, I have shown a tutorial on how to build a simple knowledge graph, store it in your database and retrieve the entity relationships for any given user query. The same thing can be extended to your RAG application to retrieve enhanced results/responses.

The only prerequisite to do this tutorial is SingleStore. Sign up & get a free account: <https://bit.ly/SingleStoreDB>

Vector RAG vs. Graph RAG



Which RAG is more superior: Graph RAG or Vector RAG?

RAG can be implemented using either a database that supports vectors and semantic search or a knowledge graph, each offering distinct advantages and methodologies for information retrieval and response generation. The goal remains the same with both approaches, to retrieve the contextually relevant data/information for the user query.

RAG with a vector database involves converting input queries into vector representations/embeddings and performing vector search to retrieve relevant data based on their semantic similarity. The retrieved documents go through an LLM to generate the responses. This approach is efficient for handling large-scale unstructured data and excels in contexts where the relationships between data points are not explicitly defined.

In contrast, RAG with a knowledge graph uses the structured relationships and entities within the graph to retrieve relevant information. The input query is used to perform a search within the knowledge graph, extracting relevant entities and their relationships.

This structured data is then utilized to generate a response. Knowledge graphs are particularly useful for applications requiring a deep understanding of the interconnections between data points, making them ideal for domains where the relationships between entities are crucial.

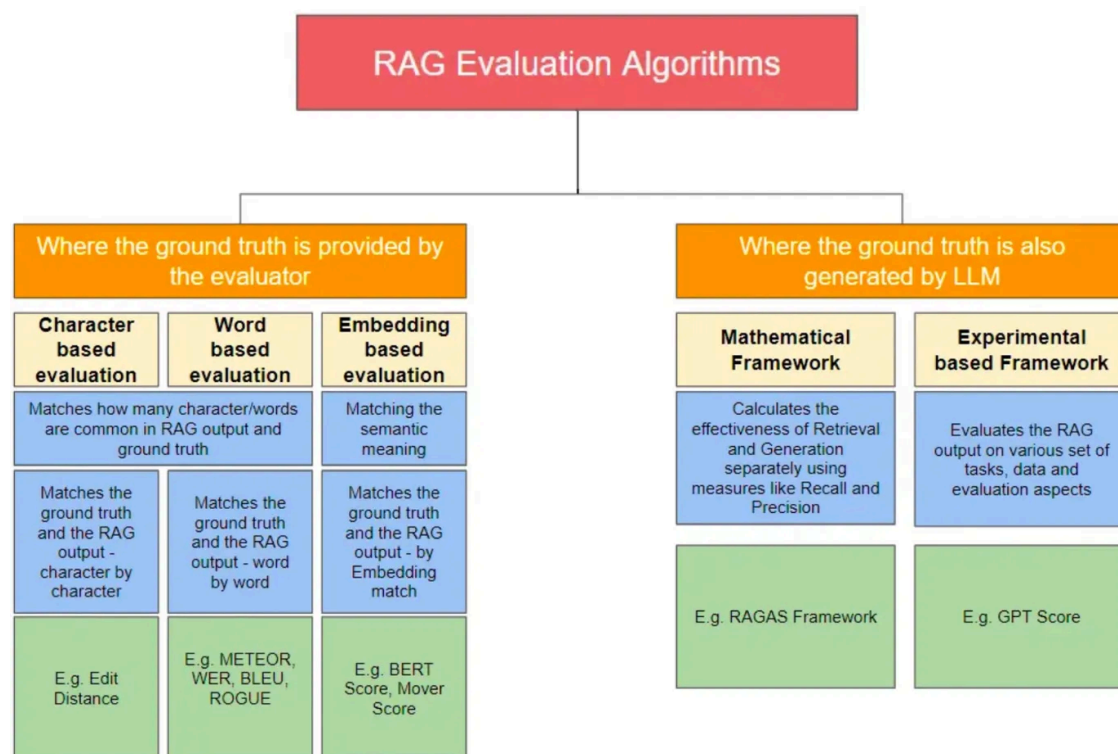
You don't need a specialised database or such to do both graph RAG or vector RAG.

Well, both approaches can be possible with SingleStore, you can use it as a vector database and also for constructing and storing knowledge graphs for graph RAG.

Try SingleStore for free: <https://bit.ly/SingleStoreDB>

Watch my recent video on enhancing RAG applications using knowledge graphs:
<https://youtu.be/rCQpQeJO59A>

RAG Evaluation Strategies



Credits: Chandan Durgla

The field of RAG evaluation continues to evolve & it is very important for AI/ML/Data engineers to know these concepts thoroughly.

RAG evaluation includes the evaluation of retrieval & the generation component with the specific input text.

At a high level, RAG evaluation algorithms can be bifurcated into two categories. 1) Where the ground truth (the ideal answer) is provided by the evaluator/user 2) Where the ground truth (the ideal answer) is also generated by another LLM.

For the ease of understanding, the author has further classified these categories into 5 sub-categories.

1. Character based evaluation

2. Word based evaluation

3. Embedding based evaluation

4. Mathematical Framework

5. Experimental based framework

Let's take a look at each of these evaluation categories:

1. Where the ground truth is provided by the evaluator.

→ **Character based evaluation algorithm:**

As the name indicates, this algorithm finds a score which is the character by character difference between the reference (ground truth) and the RAG translation output.

→ **Word based evaluation algorithm:**

As the name indicates, this algorithm finds a score which is the word by word difference between the reference (ground truth) and the RAG output.

→ **Embedding based evaluation algorithms:**

Embedding based algorithms works in two steps.

Step 1: Create embeddings for both the generated text and the reference text using a particular embedding technique

Step 2: Use a distance measure (like cosine similarity) to evaluate the distance between the embeddings of the generated text and the reference text.

2. Where the ground truth is also generated by LLM (LLM assisted evaluation)

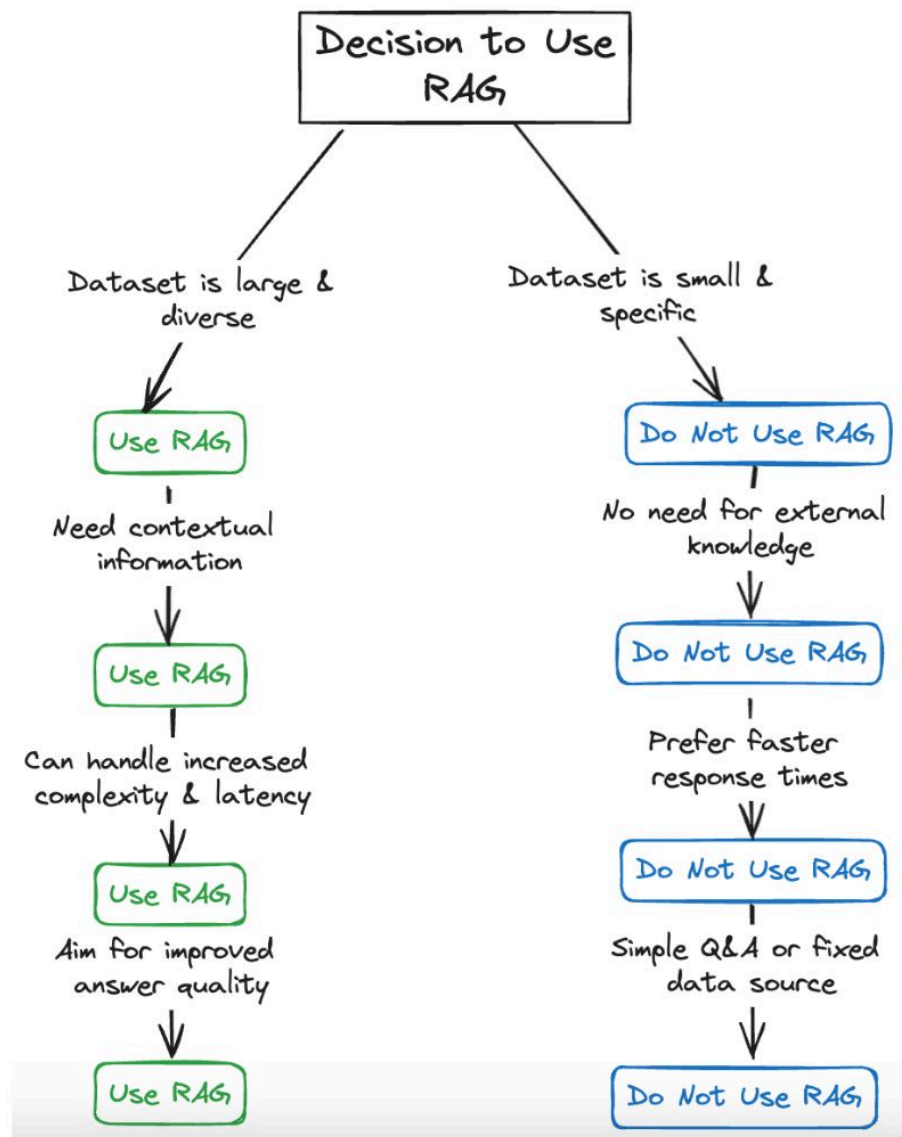
→ **Mathematical Framework — RAGAS Score**

RAGAS is one of the most common and comprehensive frameworks to assess the RAG accuracy and relevance. RAG bifurcates the evaluation from Retrieval and Generation perspective.

→ **Experimental Based Framework — GPT score**

The effectiveness of this approach in achieving desired text evaluations through natural language instructions is demonstrated by evaluating experimental results on four text generation tasks, 22 evaluation aspects, and 37 corresponding datasets. Know more about RAG evaluation in this [original article](#).

BUT....RAG Isn't a Silver Bullet :(



RAG is not a silver bullet! It's the cheapest way to improve LLMs

BUT that may not always be the case.

Here is a flowchart guiding the decision on whether to use Retrieval-Augmented Generation (RAG).

□ Dataset Size and Specificity:

If the dataset is large and diverse, proceed with considering RAG.

If the dataset is small and specific, do not use RAG.

□ For Large and Diverse Datasets:

If contextual information is needed, use RAG.

If you can handle increased complexity and latency, use RAG.
If you aim for improved search and answer quality, use RAG.

❑ For Small and Specific Datasets:

If there is no need for external knowledge, do not use RAG.

If faster response times are preferred, do not use RAG.

If the task involves simple Q&A or a fixed data source, do not use RAG.

If not RAG then what can we use? we can use fine-tuning and prompt engineering.

Fine-tuning involves training the large language model (LLM) on a specific dataset relevant to your task. This helps the LLM understand the domain and improve its accuracy for tasks within that domain.

Prompt engineering is where you focus on crafting informative prompts and instructions for the LLM. By carefully guiding the LLM with the right questions and context, you can steer it towards generating more relevant and accurate responses without needing an external information retrieval step.

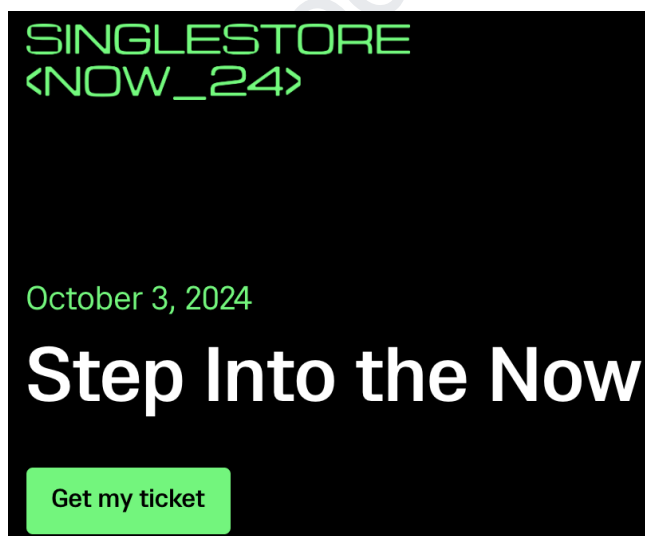
Ultimately, the best alternative depends on your specific needs.

Take a look at my article on RAG: <https://bit.ly/RAGTutorial>

If you like to use a robust database for not just AI/ML applications but also for real-time analytics, try SingleStore database.

Sign up & get free credits using my link: <https://bit.ly/SingleStoreDB>

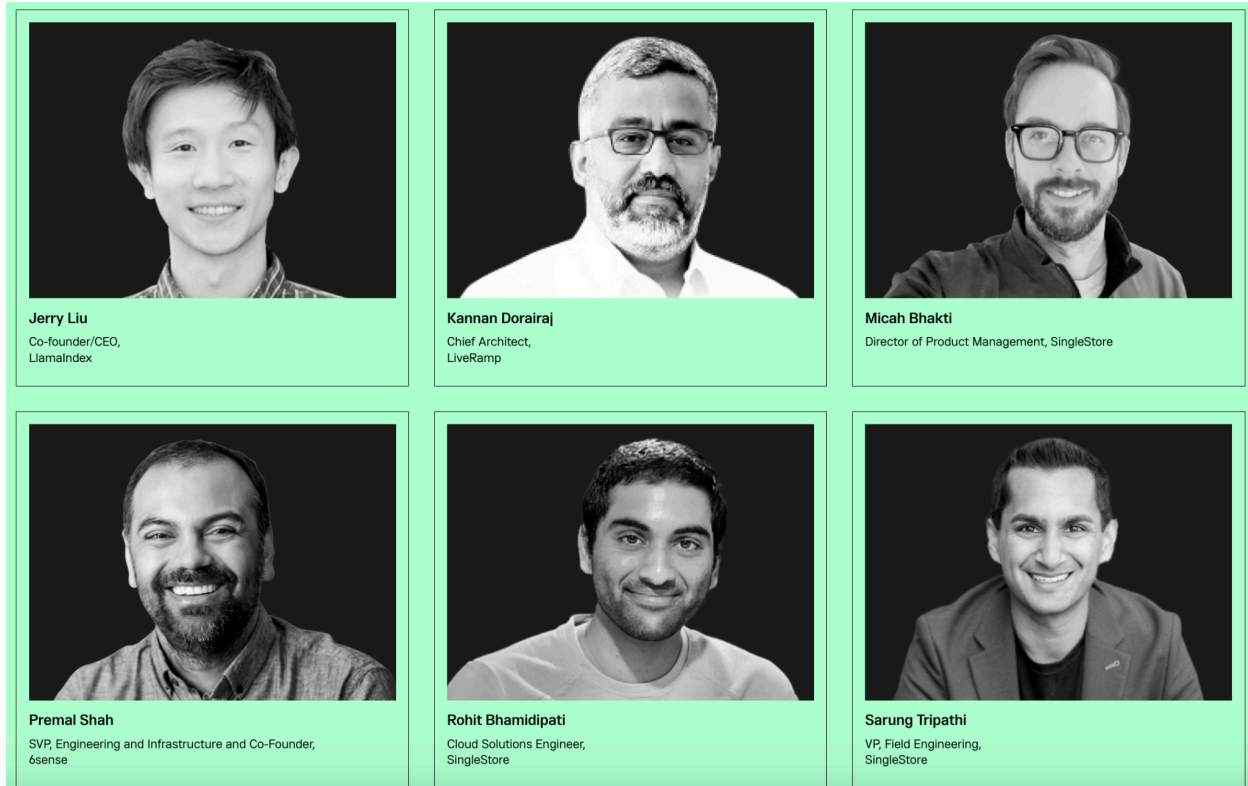
Guys, it's that time of year again, the most awaited [AI conference](#) in San Francisco, happening on the 3rd of October 2024.



If you are really interested in attending this conference where you will get to meet some great AI minds in the industry, let me know. **I have some huge discount coupons [100% free]** I can share with you.

My email address is pbelagatti@singlestore.com

I mean, it's a crazy speaker lineup, see below.



And hey, don't forget to [subscribe to my YouTube channel!](#)

Thank You!!!